

ration resource populated by the `cudaDeviceGetDevResource` API. That API can retrieve the workqueue configuration resources of a given GPU device.

- Finally, a pre-existing workqueue resource (`cudaDevResourceTypeWorkqueue`) has no fields that can be set by the user. As with the other resource types, `cudaDevGetDevResource` can retrieve the pre-existing workqueue resource of a given GPU device.

4.6.4. Green Context Creation Example

There are four main steps involved in green context creation:

- Step 1: Start with an initial set of resources, e.g., by fetching the available resources of the GPU
- Step 2: Partition the SM resources into one or more partitions (using one of the available split APIs).
- Step 3: Create a resource descriptor combining, if needed, different resources
- Step 4: Create a green context from the descriptor, provisioning its resources

After the green context has been created, you can create CUDA streams belonging to that green context. GPU work subsequently launched on such a stream, such as a kernel launched via `<<< >>>`, will only have access to this green context's provisioned resources. Libraries can also easily leverage green contexts, as long as the user passes a stream belonging to a green context to them. See [Green Contexts - Launching work](#) for more details.

4.6.4.1 Step 1: Get available GPU resources

The first step in green context creation is to get the available device resources and populate the `cudaDevResource` struct(s). There are currently three possible starting points: a device, an execution context or a CUDA stream.

The relevant CUDA runtime API function signatures are listed below:

- For a **device**: `cudaError_t cudaDeviceGetDevResource(int device, cudaDevResource* resource, cudaDevResourceType type)`
- For an **execution context**: `cudaError_t cudaExecutionContextGetDevResource(cudaExecutionContext_t ctx, cudaDevResource* resource, cudaDevResourceType type)`
- For a **stream**: `cudaError_t cudaStreamGetDevResource(cudaStream_t hStream, cudaDevResource* resource, cudaDevResourceType type)`

All valid `cudaDevResourceType` types are permitted for each of these APIs, with the exception of `cudaStreamGetDevResource` which only supports an SM-type resource.

Usually, the starting point will be a GPU device. The code snippet below shows how to get the available SM resources of a given GPU device. After a successful `cudaDeviceGetDevResource` call, the user can review the number of SMs available in this resource.

```
int current_device = 0; // assume device ordinal of 0
CUDA_CHECK(cudaSetDevice(current_device));

cudaDevResource initial_SM_resources = {};
CUDA_CHECK(cudaDeviceGetDevResource(current_device /* GPU device */,
                                   &initial_SM_resources /* device resource
→to populate */,
```

(continues on next page)

(continued from previous page)

```

        cudaDevResourceTypeSm /* resource type*/));

std::cout << "Initial SM resources: " << initial_SM_resources.sm.smCount << "
→SMs" << std::endl; // number of available SMs

// Special fields relevant for partitioning (see Step 3 below)
std::cout << "Min. SM partition size: " << initial_SM_resources.sm.
→minSmPartitionSize << " SMs" << std::endl;
std::cout << "SM co-scheduled alignment: " << initial_SM_resources.sm.
→smCoscheduledAlignment << " SMs" << std::endl;

```

One can also get the available workqueue config. resources, as shown in the code snippet below.

```

int current_device = 0; // assume device ordinal of 0
CUDA_CHECK(cudaSetDevice(current_device));

cudaDevResource initial_WQ_config_resources = {};
CUDA_CHECK(cudaDeviceGetDevResource(current_device /* GPU device */,
                                   &initial_WQ_config_resources /* device
→resource to populate */,
                                   cudaDevResourceTypeWorkqueueConfig /*
→resource type*/));

std::cout << "Initial WQ config. resources: " << std::endl;
std::cout << " - WQ concurrency limit: " << initial_WQ_config_resources.
→wqConfig.wqConcurrencyLimit << std::endl;
std::cout << " - WQ sharing scope: " << initial_WQ_config_resources.wqConfig.
→sharingScope << std::endl;

```

After a successful `cudaDeviceGetDevResource` call, the user can review the `wqConcurrencyLimit` for this resource. When the starting point is a GPU device, the `wqConcurrencyLimit` will match the value of `CUDA_DEVICE_MAX_CONNECTIONS` environment variable or its default value.

4.6.4.2 Step 2: Partition SM resources

The second step in green context creation is to statically split the available `cudaDevResource` SM resources into one or more partitions, with potentially some SMs left over in a *remaining* partition. This partitioning is possible using the `cudaDevSmResourceSplitByCount()` or the `cudaDevSmResourceSplit()` API. The `cudaDevSmResourceSplitByCount()` API can only create one or more *homogeneous* partitions, plus a potential *remaining* partition, while the `cudaDevSmResourceSplit()` API can also create *heterogeneous* partitions, plus the potential *remaining* one. The subsequent sections describe the functionality of both APIs in detail. Both APIs are only applicable to SM-type device resources.

cudaDevSmResourceSplitByCount API

The `cudaDevSmResourceSplitByCount` runtime API signature is:

```

cudaError_t cudaDevSmResourceSplitByCount(cudaDevResource* result, unsigned
int* nbGroups, const cudaDevResource* input, cudaDevResource* remaining, un-
signed int useFlags, unsigned int minCount)

```

As [Figure 46](#) highlights, the user requests to split the input SM-type device resource into `*nbGroups` homogeneous groups with `minCount` SMs each. However, the end result will contain a potentially updated `*nbGroups` number of homogeneous groups with `N` SMs each. The potentially updated

*nbGroups will be less than or equal to the originally requested group number, while N will be equal to or greater than minCount. These adjustments may occur due to some granularity and alignment requirements, which are architecture specific.

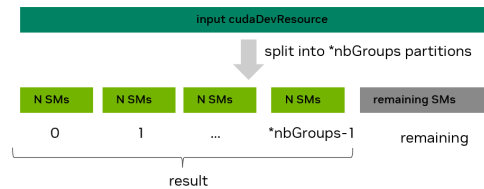


Figure 46: SM resource split using the cudaDevSmResourceSplitByCount API

Table 30 lists the minimum SM partition size and the SM co-scheduled alignment for all the currently supported compute capabilities, for the default useFlags=0 case. One can also retrieve these values via the minSmPartitionSize and smCoscheduledAlignment fields of cudaDevSmResource, as shown in [Step 1: Get available GPU resources](#). Some of these requirements can be lowered via a different useFlags value. Table 14 provides some relevant examples highlighting the difference between what is requested and the final result, along with an explanation. The table focuses on compute capability (CC 9.0), where the minimum number of SMs per partition is 8 and the SM count has to be a multiple of 8, if useFlags is zero.

Table 14: Split functionality

Re- quested			Actual (for GH200 with 132 SMs)			
	*nbGroups	min- Count	useFlags	*nbGroups with N SMs	Remain- ing SMs	Reason
2	72	0		1 group of 72 SMs	60	cannot exceed 132 SMs
6	11	0		6 groups of 16 SMs	36	multiple of 8 requirement
6	11	CU_DEV_SM_RESOURCE_SPLIT_10		6 groups with 12 SMs each	60	lowered to multiple of 2 req.
2	1	0		2 groups with 8 SMs each	116	min. 8 SMs requirement

Here is a code snippet requesting to split the available SM resources into *five groups of 8 SMs each*:

```
cudaDevResource avail_resources = {};
// Code that has populated avail_resources not shown

unsigned int min_SM_count = 8;
unsigned int actual_split_groups = 5; // may be updated

cudaDevResource actual_split_result[5] = {{}, {}, {}, {}, {}};
cudaDevResource remaining_partition = {};

CUDA_CHECK(cudaDevSmResourceSplitByCount(&actual_split_result[0],
```

(continues on next page)

(continued from previous page)

```

        &actual_split_groups,
        &avail_resources,
        &remaining_partition,
        0 /*useFlags */,
        min_SM_count));

std::cout << "Split " << avail_resources.sm.smCount << " SMs into " << actual_
→split_groups << " groups " \
    << "with " << actual_split_result[0].sm.smCount << " each " \
    << "and a remaining group with " << remaining_partition.sm.smCount <
→< " SMs" << std::endl;

```

Be aware that:

- ▶ one could use `result=nullptr` to query the number of groups that would be created
- ▶ one could set `remaining=nullptr`, if one does not care for the SMs of the remaining partition
- ▶ the *remaining* (leftover) partition does not have the same functional or performance guarantees as the homogeneous groups in *result*.
- ▶ `useFlags` is expected to be 0 in the default case, but values of `cudaDevSmResourceSplitIgnoreSmCoscheduling` and `cudaDevSmResourceSplitMaxPotentialClusterSize` are also supported
- ▶ any resulting `cudaDevResource` cannot be repartitioned without first creating a resource descriptor and a green context from it (i.e., steps 3 and 4 below)

Please refer to [cudaDevSmResourceSplitByCount](#) runtime API reference for more details.

cudaDevSmResourceSplit API

As mentioned earlier, a single `cudaDevSmResourceSplitByCount` API call can only create homogeneous partitions, i.e., partitions with the same number of SMs, plus the remaining partition. This can be limiting for heterogeneous workloads, where work running on different green contexts has different SM count requirements. To achieve heterogeneous partitions with the split-by-count API, one would usually need to re-partition an existing resource by repeating Steps 1-4 (multiple times). Or, in some cases, one may be able to create homogeneous partitions each with SM count equal to the GCD (greatest common divisor) of all the heterogeneous partitions as part of step-2 and then merge the required number of them together as part of step-3. This last approach however is not recommended, as the CUDA driver may be able to create better partitions if larger sizes were requested up front.

The `cudaDevSmResourceSplit` API aims to address these limitations by allowing the user to create non-overlapping heterogeneous partitions in a single call. The `cudaDevSmResourceSplit` runtime API signature is:

```

cudaError_t cudaDevSmResourceSplit(cudaDevResource* result, unsigned int
nbGroups, const cudaDevResource* input, cudaDevResource* remainder, unsigned
int flags, cudaDevSmResourceGroupParams* groupParams)

```

This API will attempt to partition the `input` SM-type resource into `nbGroups` valid device resources (groups) placed in the `result` array based on the requirements specified for each one in the `groupParams` array. An optional remaining partition may also be created. In a successful split, as shown in [Figure 47](#), each resource in the `result` can have a different number of SMs, but never zero SMs.

When requesting a heterogeneous split, one needs to specify the SM count (`smCount` field of relevant `groupParams` entry) for each resource in `result`. This SM count should always be a multiple of two. For the scenario in the previous image, `groupParams[0].smCount` would be X, `groupParams[1].smCount` Y, etc. However, just specifying the SM count is not sufficient, if an application uses [Thread](#)

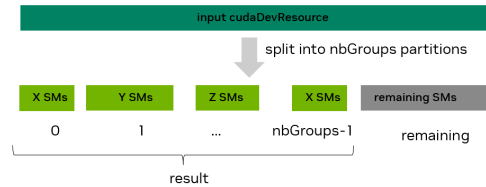


Figure 47: SM resource split using the cudaDevSmResourceSplit API

Block Clusters. Since all the thread blocks of a cluster are guaranteed to be co-scheduled, the user also needs to specify the maximum supported cluster size, if any, a given resource group should support. This is possible via the `coscheduledSmCount` field of the relevant `groupParams` entry. For GPUs with compute capability 10.0 and on (CC 10.0+), clusters can also have a preferred dimension, which is a multiple of their default cluster dimension. During a single kernel launch on supported systems, this larger preferred cluster dimension is used as much as possible, if at all, and the smaller default cluster dimension is used otherwise. The user can express this preferred cluster dimension hint via the `preferredCoscheduledSmCount` field of the relevant `groupParams` entry. Finally, there may be cases where the user may want to loosen the SM count requirements and pull in more available SMs in a given group; the user can express this backfill option by setting the `flags` field of the relevant `groupParams` entry to its non-default flag value.

To provide more flexibility, the `cudaDevSmResourceSplit` API also has a **discovery** mode, to be used when the exact SM count, for one or more groups, is not known ahead of time. For example, a user may want to create a device resource that has as many SMs as possible, while meeting some co-scheduling requirements (e.g., allowing clusters of size four). To exercise this discovery mode, the user can set the `smCount` field of the relevant `groupParams` entry (or entries) to zero. After a successful `cudaDevSmResourceSplit` call, the `smCount` field of the `groupParams` will have been populated with a valid non-zero value; we refer to this as the **actual** `smCount` value. If `result` was not null (so this was not a dry run), then the relevant group of `result` will also have its `smCount` set to the same value. The order the `nbGroups` `groupParams` entries are specified matters, as they are evaluated from left (index 0) to right (index `nbGroups-1`).

Table 15 provides a high level view of the supported arguments for the `cudaDevSmResourceSplit` API.

Table 15: Overview of `cudaDevSmResourceSplit` split API

result	nbGr	input	remain-der	groupParams array; showing entry i with i [0, nbGroups)				
				flags	smCount	cosched-uledSm-Count	preferred-Cosched-uledSm-Count	flags
nullptr for explorative dry run; not null ptr otherwise	num-ber	resource to split into nbGroups groups	nullptr if you do not want a remain-der group	0	0 for dis-covery mode or other valid sm-Count	0 (de-fault) or valid cosched-uled SM count	0 (default) or valid preferred coscheduled SM count (hint)	0 (default) or cud-aDevSm-Resource-Group-Backfill

Notes:

- 1) `cudaDevSmResourceSplit` API's return value depends on `result`:
 - ▶ `result != nullptr`: the API will return `cudaSuccess` only when the split is successful and `nbGroups` valid `cudaDevResource` groups, meeting the specified requirements were created; otherwise, it will return an error. As different types of errors may return the same error code (e.g., `CUDA_ERROR_INVALID_RESOURCE_CONFIGURATION`), it is recommended to use the `CUDA_LOG_FILE` environment variable to get more informative error descriptions during development.
 - ▶ `result == nullptr`: the API may return `cudaSuccess` even if the resulting `smCount` of a group is zero, a case which would have returned an error with a non-`nullptr` `result`. Think of this mode as a dry-run test you can use while exploring what is supported, especially in *discovery* mode.
- 2) On a successful call with `result != nullptr`, the resulting `result[i]` device resource with `i` in `[0, nbGroups)` will be of type `cudaDevResourceTypeSm` and have a `result[i].sm.smCount` that will either be the non-zero user-specified `groupParams[i].smCount` value or the discovered one. In both cases, the `result[i].sm.smCount` will meet all the following constraints:
 - ▶ be a multiple of 2 and
 - ▶ be in the `[2, input.sm.smCount]` range and
 - ▶ `(flags == 0) ? (multiple of actual group_params[i].coscheduledSmCount) : (>= groups_params[i].coscheduledSmCount)`
- 3) Specifying zero for any of the `coscheduledSmCount` and `preferredCoscheduledSmCount` fields indicates that the default values for these fields should be used; these can vary per GPU. These default values are both equal to the `smCoscheduledAlignment` of the SM resource retrieved via the `cudaDeviceGetDevResource` API for the given device (and not any SM resource). To review these default values, one can examine their updated values in the relevant `groupParams` entry after a successful `cudaDevSmResourceSplit` call with them initially set to 0; see below.

```
int gpu_device_index = 0;
cudaDevResource initial_GPU_SM_resources {};
CUDA_CHECK(cudaDeviceGetDevResource(gpu_device_index, &initial_GPU_
    ↪SM_resources, cudaDevResourceTypeSm));
std::cout << "Default value will be equal to " << initial_GPU_SM_
    ↪resources.sm.smCoscheduledAlignment << std::endl;

int default_split_flags = 0;
cudaDevSmResourceGroupParams group_params_tmp = {.smCount=0, .
    ↪coscheduledSmCount=0, .preferredCoscheduledSmCount=0, .flags=0};
CUDA_CHECK(cudaDevSmResourceSplit(nullptr, 1, &initial_GPU_SM_
    ↪resources, nullptr /*remainder*/, default_split_flags, &group_
    ↪params_tmp));
std::cout << "coscheduledSmcount default value: " << group_params.
    ↪coscheduledSmCount << std::endl;
std::cout << "preferredCoscheduledSmcount default value: " << group_
    ↪params.preferredCoscheduledSmCount << std::endl;
```

- 4) The remainder group, if present, will not have any constraints on its SM count or co-scheduling requirements. It will be up to the user to explore that.

Before providing more detailed information for the various `cudaDevSmResourceGroupParams` fields, Table 16 shows what these values could be for some example use cases. Assume an `initial_GPU_SM_resources` device resource has already been populated, as in the previous code snippet, and is the resource that will be split. Every row in the table will have that same starting point. For

simplicity the table will only show the nbGroups value and the groupParams fields per use case that can be used in a code snippet like the one below.

```
int nbGroups = 2; // update as needed
unsigned int default_split_flags = 0;
cudaDevResource remainder {}; // update as needed
cudaDevResource result_use_case[2] = {{}, {}}; // Update depending on number
→ of groups planned. Increase size if you plan to also use a workqueue resource
cudaDevSmResourceGroupParams group_params_use_case[2] = {{.smCount = X, .
→ coscheduledSmCount=0, .preferredCoscheduledSmCount = 0, .flags = 0},
→ { .smCount = Y, .
→ coscheduledSmCount=0, .preferredCoscheduledSmCount = 0, .flags = 0}}
CUDA_CHECK(cudaDevSmResourceSplit(&result_use_case[0], nbGroups, &initial_GPU_
→ SM_resources, remainder, default_split_flags, &group_params_use_case[0]));
```

Table 16: split API use cases

#	Goal/Use Cases	nbGr	re- main- der	groupParams[i] fields (i shown in ascending order; see last column)				flags	i
				sm- Count	cosched- uledSm- Count	preferred- Cosched- uledSm- Count			
1	Resource with 16 SMs. Do not care for remaining SMs. May use clusters.	1	nullp	16	0	0		0	0
2a	One resource with 16 SMs and one with everything else. Will not use clusters.	1	not nullp	16	2	2		0	0
2b	(Note: showing two options: in option (2a), the 2nd resource is the remainder; in option (2b), it is the result_use_case[1].)	2	nullp	16	2	2		0	0
				0	2	2		cudaDevSmResourceGroupBackfill	1
3	Two resources with 28 and 32 SMs respectively. Will use clusters of size 4.	2	nullp	28	4	4		0	0
				32	4	4		0	1
4	One resource with as many SMs as possible, which can run clusters of size 8, and one remainder.	1	not nullp	0	8	8		0	0
5	One resource with as many SMs as possible, which can run clusters of size 4, and one with 8 SMs. (Note: Order matters! Changing order of entries in groupParams array could mean no SMs left for the 8-SM group)	2	nullp	8	2	2		0	0
				0	4	4		0	1

Detailed information about the various cudaDevSmResourceGroupParams struct fields

smCount:

- Controls SM count for the corresponding group in result.
- **Values:** 0 (discovery mode) or valid non-zero value (non-discovery mode)

- ▶ Valid non-zero `smCount` value requirements: (multiple of 2) and in `[2, input->sm.smCount]` and `((flags == 0) ? multiple of actual coscheduledSmCount : greater than or equal to coscheduledSmCount)`
- ▶ **Use cases:** use discovery mode to explore what's possible when SM count is not known/fixed; use non-discovery mode to request a specific number of SMs.
- ▶ Note: in discovery mode, actual SM count, after successful split call with non-nullptr result, will meet valid non-zero value requirements

`coscheduledSmCount`:

- ▶ Controls number of SMs grouped together (“co-scheduled”) to enable launch of different clusters on compute capability 9.0+. It can thus impact the number of SMs in a resulting group and the cluster sizes they can support.
- ▶ **Values:** 0 (default for current architecture) or valid non-zero value
 - ▶ Valid non-zero value requirements: (multiple of 2) up to max limit
- ▶ **Use cases:** Use default or a manually chosen value for clusters, keeping in mind the max. portable cluster size on a given architecture. If your code does not use clusters, you can use the minimum supported value of 2 or the default value.
- ▶ Note: when the default value is used, the actual `coscheduledSmCount`, after a successful split call, will also meet valid non-zero value requirements. If `flags` is not zero, the resulting `smCount` will be $\geq$ `coscheduledSmCount`. Think of `coscheduledSmCount` as providing some guaranteed underlying “structure” to valid resulting groups (i.e., that group can run at least a single cluster of `coscheduledSmCount` size in the worst case). This type of structure guarantee does not apply to the remaining group; there it is up to the user to explore what cluster sizes can be launched.

`preferredCoscheduledSmCount`:

- ▶ Acts as a hint to the driver to try to merge groups of actual `coscheduledSmCount` SMs into larger groups of `preferredCoscheduledSmCount` if possible. Doing so can allow code to make use of preferred cluster dimensions feature available on devices with compute capability (CC) 10.0 and on). See [cudaLaunchAttributeValue::preferredClusterDim](#).
- ▶ **Values:** 0 (default for current architecture) or valid non-zero value
 - ▶ Valid non-zero value requirements: (multiple of actual `coscheduledSmCount`)
- ▶ **Use cases:** use a manually chosen value greater than 2 if you use preferred clusters and are on a device of compute capability 10.0 (Blackwell) or later. If you don't use clusters, choose the same value as `coscheduledSmCount`: either select the minimum supported value of 2 or use 0 for both
- ▶ Note: when the default value is used, the actual `preferredCoscheduledSmCount`, after a successful split call, will also meet valid non-zero value requirement.

`flags`:

- ▶ Controls if the resulting SM count of a group will be a multiple of actual coscheduled SM count (default) or if SMs can be backfilled into this group (backfill). In the backfill case, the resulting SM count (`result[i].sm.smCount`) will be greater than or equal to the specified `groupParams[i].smCount`.
- ▶ **Values:** 0 (default) or `cudaDevSmResourceGroupBackfill`
- ▶ **Use cases:** Use the zero (default), so the resulting group has the guaranteed flexibility of supporting multiple clusters of `coScheduledSmCount` size. Use the backfill option, if you want to get as many SMs as possible in the group, with some of these SMs (the backfilled ones), not providing any coscheduling guarantee.

- Note: a group created with the backfill flag can still support clusters (e.g., it is guaranteed to support at least one coscheduledSmCount size).

4.6.4.3 Step 2 (continued): Add workqueue resources

If you also want to specify a workqueue resource, then this needs to be done explicitly. The following example shows how to create a workqueue configuration resource for a specific device with balanced sharing scope and a concurrency limit of four.

```
cudaDevResource split_result[2] = {{}}, {{}};
// code to populate split_result[0] not shown; used split API with nbGroups=1

// The last resource will be a workqueue resource.
split_result[1].type = cudaDevResourceTypeWorkqueueConfig;
split_result[1].wqConfig.device = 0; // assume device ordinal of 0
split_result[1].wqConfig.sharingScope =
    ↪ cudaDevWorkqueueConfigScopeGreenCtxBalanced;
split_result[1].wqConfig.wqConcurrencyLimit = 4;
```

A workqueue concurrency limit of four hints to the driver that the user expects maximum four concurrent stream-ordered workloads. The driver will assign work queues trying to respect this hint, if possible.

4.6.4.4 Step 3: Create a Resource Descriptor

The next step, after resources have been split, is to generate a resource descriptor, using the `cudaDevResourceGenerateDesc` API, for all the resources expected to be available to a green context.

The relevant CUDA runtime API function signature is:

```
cudaError_t cudaDevResourceGenerateDesc(cudaDevResourceDesc_t *phDesc, cudaDevResource *resources, unsigned int nbResources)
```

It is possible to combine multiple `cudaDevResource` resources. For example, the code snippet below shows how to generate a resource descriptor that encapsulates three groups of resources. You just need to ensure that these resources are all allocated continuously in the `resources` array.

```
cudaDevResource actual_split_result[5] = {};
// code to populate actual_split_result not shown

// Generate resource desc. to encapsulate 3 resources: actual_split_result[2]
    ↪ to [4]
cudaDevResourceDesc_t resource_desc;
CUDA_CHECK(cudaDevResourceGenerateDesc(&resource_desc, &actual_split_
    ↪ result[2], 3));
```

Combining different types of resources is also supported. For example, one could generate a descriptor with both SM and workqueue resources.

For a `cudaDevResourceGenerateDesc` call to be successful:

- All `nbResources` resources should belong to the same GPU device.
- If multiple SM-type resources are combined, they should be generated from the same split API call and have the same `coscheduledSmCount` values (if not part of remainder)
- Only a single workqueue config or workqueue type resource may be present.

4.6.4.5 Step 4: Create a Green Context

The final step is to create a green context from a resource descriptor using the `cudaGreenCtxCreate` API. That green context will only have access to the resources (e.g., SMs, work queues) encapsulated in the resource descriptor specified during its creation. These resources will be provisioned during this step.

The relevant CUDA runtime API function signature is:

```
cudaError_t cudaGreenCtxCreate(cudaExecutionContext_t *phCtx, cudaDevResourceDesc_t desc, int device, unsigned int flags)
```

The `flags` parameter should be set to 0. It is also recommended to explicitly initialize the device's primary context before creating a green context via either the `cudaInitDevice` API or the `cudaSetDevice` API, which also sets the primary context as current to the calling thread. Doing so ensures there will be no additional primary context initialization overhead during green context creation.

See code snippet below.

```
int current_device = 0; // assume single GPU
CUDA_CHECK(cudaSetDevice(current_device)); // Or cudaInitDevice

cudaDevResourceDesc_t resource_desc {};
// Code to generate resource_desc not shown

// Create a green_ctx on GPU with current_device ID with access to resources
// from resource_desc
cudaExecutionContext_t green_ctx {};
CUDA_CHECK(cudaGreenCtxCreate(&green_ctx, resource_desc, current_device, 0));
```

After a successful green context creation, the user can verify its resources by calling `cudaExecutionContextGetDevResource` on that execution context for each resource type.

Creating Multiple Green Contexts

An application can have more than one green context, in which case some of the steps above should be repeated. For most use cases, these green contexts will each have a separate non-overlapping set of provisioned SMs. For example, for the case of five homogeneous `cudaDevResource` groups (`actual_split_result` array), one green context's descriptor may encapsulate `actual_split_result[2]` to `[4]` resources, while the descriptor of another green context may encapsulate `actual_split_result[0]` to `[1]`. In this case, a specific SM will be provisioned for only one of the two green contexts of the application.

But SM oversubscription is also possible and may be used in some cases. For example, it may be acceptable to have the second green context's descriptor encapsulate `actual_split_result[0]` to `[2]`. In this case, all the SMs of `actual_split_resource[2]` `cudaDevResource` will be oversubscribed, i.e., provisioned for both green contexts, while resources `actual_split_resource[0]` to `[1]` and `actual_split_resource[3]` to `[4]` may only be used by one of the two green contexts. SM oversubscription should be judiciously used on a per-case basis.

4.6.5. Green Contexts - Launching work

To launch a kernel targeting a green context created using the prior steps, you first need to create a stream for that green context with the `cudaExecutionContextStreamCreate` API. Launching a kernel on that stream using `<<< >>>` or the `cudaLaunchKernel` API, will ensure that kernel can only use the resources (SMs, work queues) available to that stream via its execution context. For example:

```
// Create green_ctx_stream CUDA stream for previously created green_ctx green
↳context
cudaStream_t green_ctx_stream;
int priority = 0;
CUDA_CHECK(cudaExecutionCtxStreamCreate(&green_ctx_stream,
                                         green_ctx,
                                         cudaStreamDefault,
                                         priority));

// Kernel my_kernel will only use the resources (SMs, work queues, as
↳applicable) available to green_ctx_stream's execution context
my_kernel<<<grid_dim, block_dim, 0, green_ctx_stream>>>();
CUDA_CHECK(cudaGetLastError());
```

The default stream creation flag passed to the stream creation API above is equivalent to `cudaStreamNonBlocking` given `green_ctx` is a green context.

CUDA graphs

For kernels launched as part of a CUDA graph (see [CUDA Graphs](#)), there are a few more subtleties. Unlike kernels, the CUDA stream a CUDA graph is launched on does **not** determine the SM resources used, as that stream is solely used for dependency tracking.

The execution context a kernel node (and other applicable node types) will execute on is set during node creation. If the CUDA graph will be created using stream capture, then the execution context(s) of the stream(s) involved in the capture will determine the execution context(s) of the relevant graph nodes. If the graph will be created using the graph APIs, then the user should explicitly set the execution context for each relevant node. For example, to add a kernel node, the user should use the polymorphic `cudaGraphAddNode` API with `cudaGraphNodeTypeKernel` type and explicitly specify the `.ctx` field of the `cudaKernelNodeParamsV2` struct under `.kernel`. The `cudaGraphAddKernelNode` does not allow the user to specify an execution context and should thus be avoided. Please note that it is possible for different graph nodes in a graph to belong to different execution contexts.

For verification purposes, one could use Nsight Systems in node tracing mode (`--cuda-graph-trace node`) to observe the green context(s) specific graph nodes will execute on. Note that in the default *graph* tracing mode, the entire graph will appear under the green context of the stream it was launched on, but, as previously explained, this does not provide any information about the execution context(s) of the various graph nodes.

To verify programmatically, one could potentially use the CUDA driver API `cuGraphKernelNodeGetParams(graph_node, &node_params)` and compare the `node_params.ctx` context handle field with the expected context handle for that graph node. Using the driver API is possible given `CUgraphNode` and `cudaGraphNode_t` can be used interchangeably, but the user would need to include the relevant `cuda.h` header and link with the driver directly (`-lcuda`).

Thread Block Clusters

Kernels with thread block clusters (see [Section 1.2.2.1.1](#)) can also be launched on a green context stream, like any other kernel, and thus use that green context's provisioned resources. [Section 4.6.4.2](#) showed how to specify the number of SMs that need to be coscheduled when a device resource is split, to facilitate clusters. But as with any kernel using clusters, the user should make use of the relevant occupancy APIs to determine the max potential cluster size for a kernel (via `cudaOccupancyMaxPotentialClusterSize`) and, if needed, the maximum number of active clusters (`cudaOccupancyMaxActiveClusters`). If the user specifies a green context stream as the `stream` field of the relevant `cudaLaunchConfig`, then these occupancy APIs will take into consideration the SM resources provisioned for that green context. This use case is especially relevant for libraries that

may get a green context CUDA stream passed to them by the user, as well as in cases where the green context was created from a remaining device resource.

The code snippet below shows how these APIs can be used.

```
// Assume cudaStream_t gc_stream has already been created and a __global__
→void cluster_kernel exists.

// Uncomment to support non portable cluster size, if possible
// CUDA_CHECK(cudaFuncSetAttribute(cluster_kernel,
→cudaFuncAttributeNonPortableClusterSizeAllowed, 1))

cudaLaunchConfig_t config = {0};
config.gridDim          = grid_dim; // has to be a multiple of cluster dim.
config.blockDim         = block_dim;
config.dynamicSmemBytes = expected_dynamic_shared_mem;

cudaLaunchAttribute attribute[1];
attribute[0].id = cudaLaunchAttributeClusterDimension;
attribute[0].val.clusterDim.x = 1;
attribute[0].val.clusterDim.y = 1;
attribute[0].val.clusterDim.z = 1;
config.attrs = attribute;
config.numAttrs = 1;

config.stream=gc_stream; // Need to pass the CUDA stream that will be used for
→that kernel

int max_potential_cluster_size = 0;
// the next call will ignore cluster dims in launch config
CUDA_CHECK(cudaOccupancyMaxPotentialClusterSize(&max_potential_cluster_size,
→cluster_kernel, &config));
std::cout << "max potential cluster size is " << max_potential_cluster_size <
→< " for CUDA stream gc_stream" << std::endl;

// Could choose to update launch config's clusterDim with max_potential_cluster_
→size.
// Doing so would result in a successful cudaLaunchKernelEx call for the same
→kernel and launch config.

int num_clusters= 0;
CUDA_CHECK(cudaOccupancyMaxActiveClusters(&num_clusters, cluster_kernel, &
→config));
std::cout << "Potential max. active clusters count is " << num_clusters <<
→std::endl;
```

Verify Green Contexts Use

Beyond empirical observations of affected kernel execution times due to green context provisioning, the user can leverage [Nsight Systems](#) or [Nsight Compute](#) CUDA developer tools to verify, to some extent, correct green contexts use.

For example, kernels launched on CUDA streams belonging to different green contexts will appear under different *Green Context* rows under the CUDA HW timeline section of an *Nsight Systems* report. *Nsight Compute* provides a *Green Context Resources* overview in its Session page as well as updated # SMs under the *Launch Statistics* of the *Details* section. The former provides a visual bitmask of

provisioned resources. This is particularly useful if an application uses different green contexts, as the user can confirm expected overlap across GCs (no overlap or expected non-zero overlap if SMs are oversubscribed).

Figure 48 depicts these resources for an example with two green contexts provisioned with 112 and 16 SMs respectively, with no SM overlap across them. The provided view can help the user verify the provisioned SM resource count per green context. It also helps confirm that no SMs were oversubscribed, as no box is marked green (provisioned for that GC) across both green contexts.

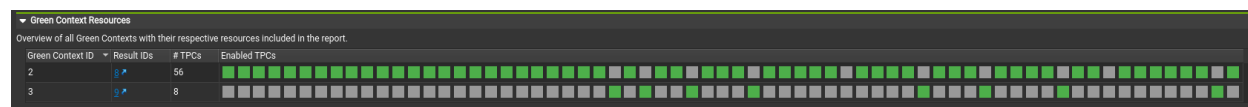


Figure 48: Green contexts resources section from Nsight Compute

The *Launch Statistics* section also explicitly lists the number of SMs provisioned for this green context, which can thus be used by this kernel. Please note that these are the SMs a given kernel can have access to during its execution, and not the actual number of SMs that kernel ran on. The same applies to the resources overview shown earlier. The actual number of SMs used by the kernel can depend on various factors, including the kernel itself (launch geometry, etc.), other work running at the same time on the GPU, etc.

4.6.6. Additional Execution Contexts APIs

This section touches upon some additional green context APIs. For a complete list, please refer to the relevant CUDA runtime API [section](#).

For synchronization using CUDA events, one can leverage the `cudaError_t cudaExecutionCtxRecordEvent(cudaExecutionContext_t ctx, cudaEvent_t event)` and `cudaError_t cudaExecutionCtxWaitEvent(cudaExecutionCtxWaitEvent(cudaExecutionContext_t ctx, cudaEvent_t event))` APIs. `cudaExecutionCtxRecordEvent` records a CUDA event capturing all work/activities of the specified execution context at the time of this call, while `cudaExecutionCtxWaitEvent` makes all future work submitted to the execution context wait for the work captured in the specified event.

Using `cudaExecutionCtxRecordEvent` is more convenient than `cudaEventRecord` if the execution context has multiple CUDA streams. To achieve equivalent behavior without this execution context API, one would need to record a separate CUDA event via `cudaEventRecord` on every execution context stream and then have dependent work wait separately for all these events. Similarly, `cudaExecutionCtxWaitEvent` is more convenient than `cudaStreamWaitEvent`, if one needs all execution context streams to wait for an event to complete. The alternative would be a separate `cudaStreamWaitEvent` for every stream in this execution context.

For blocking synchronization on the CPU side, one can use `cudaError_t cudaExecutionCtxSynchronize(cudaExecutionContext_t ctx)`. This call will block until the specified execution context has completed all its work. If the specified execution context was not created via `cudaGreenCtxCreate`, but was rather obtained via `cudaDeviceGetExecutionCtx`, and is thus the device's primary context, calling that function will also synchronize all green contexts that have been created on the same device.

To retrieve the device a given execution context is associated with, one can use `cudaExecutionCtxGetDevice`. To retrieve the unique identifier of a given execution context, one can use `cudaExecutionCtxGetId`.

Finally, an explicitly created execution context can be destroyed via the `cudaError_t cudaExecutionCtxDestroy(cudaExecutionContext_t ctx)` API.

4.6.7. Green Contexts Example

This section illustrates how green contexts can enable critical work to start and complete sooner. Similar to the scenario used in [Section 4.6.1](#), the application has two kernels that will run on two different non-blocking CUDA streams. The timeline, from the CPU side, is as follows. A long running kernel (*delay_kernel_us*), which takes multiple waves on the full GPU, is launched first on CUDA stream *strm1*. Then after a brief wait time (less than the kernel duration), a shorter but critical kernel (*critical_kernel*) is launched on stream *strm2*. The GPU durations and time from CPU launch to completion for both kernels are measured.

As a proxy for a long running kernel, a delay kernel is used where every thread block runs for a fixed number of microseconds and the number of thread blocks exceeds the GPU's available SMs.

Initially, no green contexts are used, but the critical kernel is launched on a CUDA stream with a higher priority than the long running kernel. Because of its high priority stream, the critical kernel can start executing as soon as some of the thread blocks of the long running kernel complete. However, it will still need to wait for some potentially long-running thread blocks to complete, which will delay its execution start.

[Figure 49](#) shows this scenario in an Nsight Systems report. The long running kernel is launched on stream 13, while the short but critical kernel is launched on stream 14, which has higher stream priority. As highlighted on the image, the critical kernel waits for 0.9ms (in this case) before it can start executing. If the two streams had identical priorities, the critical kernel would execute much later.

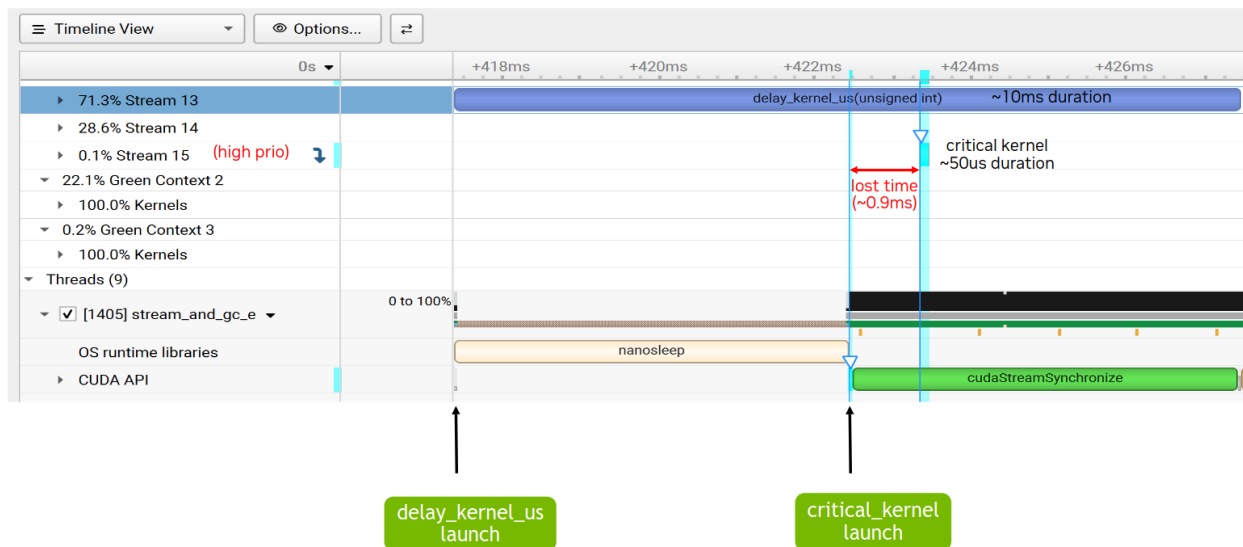
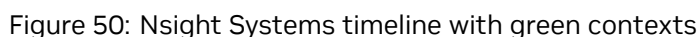


Figure 49: Nsight Systems timeline without green contexts

To leverage the green contexts feature, two green contexts are created, each provisioned with a distinct non-overlapping set of SMs. The exact SM split in this case for an H100 with 132 SMs was chosen, for illustration purposes, as 16 SMs for the critical kernel (Green Context 3) and 112 SMs for the long running kernel (Green Context 2). As [Figure 50](#) shows, the critical kernel can now start almost instantaneously, as there are SMs only Green Context 3 can use.

The duration of the short kernel may increase, compared to its duration when running in isolation, as there is now a limit on the number of SMs it can use. The same is also the case for the long running kernel, which can no longer use all the SMs of the GPU, but is constrained by its green context's provisioned resources. However, the key result is that the critical kernel work can now start and complete significantly sooner than before. That is barring any other limitations, as parallel execution, as



In all cases, the exact SM split should be decided on a per case basis after experimentation.

4.7.1. Introduction

4.7.2. Change History

CUDA Version	Change
12.3	Lazy loading performance improved. Now enabled by default for Windows.
12.2	Lazy loading enabled by default for Linux.
11.7	Lazy loading first introduced, disabled by default.

Lazy loading is a joint feature of both the CUDA runtime and driver. Lazy loading is only available when the runtime and driver version requirements are satisfied.

4.7.3.1 CUDA Runtime Version Requirement

Lazy loading is available starting in CUDA runtime version 11.7. As CUDA runtime is usually linked statically into programs and libraries, only programs and libraries from or compiled with CUDA 11.7+ toolkit will benefit from lazy loading. Libraries compiled using older CUDA runtime versions will load all modules eagerly.

4.7.3.2 CUDA Driver Version Requirement

Lazy loading requires driver version 515 or newer. Lazy loading is not available for driver versions older than 515, even when using CUDA toolkit 11.7 or newer.

4.7.3.3 Compiler Requirements

Lazy loading does not require any compiler support. Both SASS and PTX compiled with pre-11.7 compilers can be loaded with lazy loading enabled, and will see full benefits of the feature. However, the version 11.7+ CUDA runtime is still required, as described above.

4.7.3.4 Kernel Requirements

Lazy loading does not affect modules containing managed variables, which will still be loaded eagerly.

4.7.4. Usage

4.7.4.1 Enabling & Disabling

Lazy loading is enabled by setting the `CUDA_MODULE_LOADING` environment variable to `LAZY`. Lazy loading can be disabled by setting the `CUDA_MODULE_LOADING` environment variable to `EAGER`. As of CUDA 12.3, lazy loading is enabled by default on all platforms.

4.7.4.2 Checking if Lazy Loading is Enabled at Runtime

The `cuModuleGetLoadingMode` API in the CUDA driver API can be used to determine if lazy loading is enabled. Note that CUDA must be initialized before running this function. Sample usage is shown in the snippet below.

```
#include "<cuda.h>"
#include "<assert.h>"
#include "<iostream>"

int main() {
    CUmoduleLoadingMode mode;

    assert(CUDA_SUCCESS == cuInit(0));
    assert(CUDA_SUCCESS == cuModuleGetLoadingMode(&mode));

    std::cout << "CUDA Module Loading Mode is " << ((mode == CU_MODULE_
    ↪LAZY_LOADING) ? "lazy" : "eager") << std::endl;

    return 0;
}
```

4.7.4.3 Forcing a Module to Load Eagerly at Runtime

Loading kernels and variables happens automatically, without any need for explicit loading. Kernels can be loaded explicitly even without executing them by doing the following:

- ▶ The `cuModuleGetFunction()` function will cause a module to be loaded into device memory
- ▶ The `cudaFuncGetAttributes()` function will cause a kernel to be loaded into device memory

Note

`cuModuleLoad()` does not guarantee that a module will be loaded immediately.

4.7.5. Potential Hazards

Lazy loading is designed so that it should not require any modifications to applications to use it. That said, there are some caveats, especially when applications are not fully compliant with the CUDA programming model, as described below.

4.7.5.1 Impact on Concurrent Kernel Execution

Some programs incorrectly assume that concurrent kernel execution is guaranteed. A deadlock can occur if cross-kernel synchronization is required, but kernel execution has been serialized. To minimize the impact of lazy loading on concurrent kernel execution, do the following:

- ▶ preload all kernels that you hope to execute concurrently prior to launching them or
- ▶ run application with `CUDA_MODULE_LOADING = EAGER` to force loading data eagerly without forcing each function to load eagerly

4.7.5.2 Large Memory Allocations

Lazy loading delays memory allocation for CUDA modules from program initialization until closer to execution time. If an application allocates the entire VRAM on startup, CUDA can fail to allocate memory for modules at runtime. Possible solutions:

- ▶ use `cudaMallocAsync()` instead of an allocator that allocates the entire VRAM on startup
- ▶ add some buffer to compensate for the delayed loading of kernels
- ▶ preload all kernels that will be used in the program before trying to initialize the allocator

4.7.5.3 Impact on Performance Measurements

Lazy loading may skew performance measurements by moving CUDA module initialization into the measured execution window. To avoid this:

- ▶ do at least one warmup iteration prior to measurement
- ▶ preload the benchmarked kernel prior to launching it

4.8. Error Log Management

The *Error Log Management* mechanism allows for CUDA API errors to be reported to developers in a plain-English format that describes the cause of the issue.

4.8.1. Background

Traditionally, the only indication of a failed CUDA API call is the return of a non-zero code. As of CUDA Toolkit 12.9, the CUDA Runtime defines over 100 different return codes for error conditions, but many of them are generic and give the developer no assistance with debugging the cause.

4.8.2. Activation

Set the `CUDA_LOG_FILE` environment variable. Acceptable values are `stdout`, `stderr`, or a valid path on the system to write a file. The log buffer can be dumped via API even if `CUDA_LOG_FILE` was not set before program execution. Note that an error-free execution might not print any logs.

4.8.3. Output

Logs are output in the following format:

```
[Time][TID][Source][Severity][API Entry Point] Message
```

The following line is an actual error message that is generated if the developer tries to dump the Error Log Management logs to an unallocated buffer:

```
[22:21:32.099][25642][CUDA][E][cuLogsDumpToMemory] buffer cannot be NULL
```

Where before, all the developer would have gotten is `CUDA_ERROR_INVALID_VALUE` in the return code and possibly “invalid argument” if `cuGetErrorString` is called.

4.8.4. API Description

The CUDA Driver provides APIs in two categories for interacting with the Error Log Management feature.

This feature allows developers to register callback functions to be used whenever an error log is generated, where the callback signature is:

```
void callbackFunc(void *data, CUlogLevel logLevel, char *message, size_t
↪length)
```

Callbacks are registered with this API:

```
CUresult cuLogsRegisterCallback(CUlogsCallback callbackFunc, void *userData,
↪CUlogsCallbackHandle *callback_out)
```

Where `userData` is passed to the callback function without modifications. `callback_out` should be stored by the caller for use in `cuLogsUnregisterCallback`.

```
CUresult cuLogsUnregisterCallback(CUlogsCallbackHandle callback)
```

The other set of API functions are for managing the output of logs. An important concept is the log iterator, which points to the current end of the buffer:

```
CUresult cuLogsCurrent(CUlogIterator *iterator_out, unsigned int flags)
```

The iterator position can be kept by the calling software in situations where a dump of the entire log buffer is not desired. Currently, the flags parameter must be 0, with additional options reserved for future CUDA releases.

At any time, the error log buffer can be dumped to either a file or memory with these functions:

```
CUresult cuLogsDumpToFile(CUlogIterator *iterator, const char *pathToFile,  
    ↪ unsigned int flags)  
CUresult cuLogsDumpToMemory(CUlogIterator *iterator, char *buffer, size_t  
    ↪ *size, unsigned int flags)
```

If *iterator* is NULL, the entire buffer will be dumped, up to the maximum of 100 entries. If *iterator* is not NULL, logs will be dumped starting from that entry and the value of *iterator* will be updated to the current end of the logs, as if *cuLogsCurrent* had been called. If there have been more than 100 log entries into the buffer, a note will be added at the start of the dump noting this.

The flags parameter must be 0, with additional options reserved for future CUDA releases.

The *cuLogsDumpToMemory* function has additional considerations:

1. The buffer itself will be null-terminated, but each individual log entry will only be separated by a newline (n) character.
2. The maximum size of the buffer is 25600 bytes.
3. If the value provided in *size* is not sufficient to store all desired logs, a note will be added as the first entry and the oldest entries that do not fit will not be dumped.
4. After returning, *size* will contain the actual number of bytes written to the provided buffer.

4.8.5. Limitations and Known Issues

1. The log buffer is limited to 100 entries. After this limit is reached, the oldest entries will be replaced and log dumps will contain a line noting the rollover.
2. Not all CUDA APIs are covered yet. This is an ongoing project to provide better usage error reporting for all APIs.
3. The Error Log Management log location (if given) will not be tested for validity until/unless a log is generated.
4. The Error Log Management APIs are currently only available via the CUDA Driver. Equivalent APIs will be added to the CUDA Runtime in a future release.
5. The log messages are not localized to any language and all provided logs are in US English.

4.9. Asynchronous Barriers

Asynchronous barriers, introduced in *Advanced Synchronization Primitives*, extend CUDA synchronization beyond `__syncthreads()` and `__syncwarp()`, enabling fine-grained, non-blocking coordination and better overlap of communication and computation.

This section provides details on how to use asynchronous barriers mainly via the `cuda::barrier` API (with pointers to `cuda::ptx` and primitives where applicable).

4.9.1. Initialization

Initialization must happen before any thread begins participating in a barrier.

CUDA C++ `cuda::barrier`

```
#include <cuda/barrier>
#include <cooperative_groups.h>

__global__ void init_barrier()
{
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        // A single thread initializes the total expected arrival count.
        init(&bar, block.size());
    }
    block.sync();
}
```

CUDA C++ `cuda::ptx`

```
#include <cuda/ptx>
#include <cooperative_groups.h>

__global__ void init_barrier()
{
    __shared__ uint64_t bar;
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        // A single thread initializes the total expected arrival count.
        cuda::ptx::mbarrier_init(&bar, block.size());
    }
    block.sync();
}
```

CUDA C primitives

```
#include <cuda_aware_barrier_primitives.h>
#include <cooperative_groups.h>

__global__ void init_barrier()
{
    __shared__ uint64_t bar;
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        // A single thread initializes the total expected arrival count.
        __mbarrier_init(&bar, block.size());
    }
    block.sync();
}
```

Before any thread can participate in a barrier, the barrier must be initialized using the `cuda::barrier::init()` friend function. This must happen before any thread arrives on the barrier. This poses a bootstrapping challenge in that threads must synchronize before participating in the barrier, but threads are creating a barrier in order to synchronize. In this example, threads that will participate are part of a cooperative group and use `block.sync()` to bootstrap initialization. Since a whole thread block is participating in the barrier, `__syncthreads()` could also be used.

The second parameter of `init()` is the *expected arrival count*, i.e., the number of times `bar.arrive()` will be called by participating threads before a participating thread is unblocked from its call to `bar.wait(std::move(token))`. In this and the previous examples, the barrier is initialized with the number of threads in the thread block i.e., `cooperative_groups::this_thread_block().size()`, so that all threads within the thread block can participate in the barrier.

Asynchronous barriers are flexible in specifying *how* threads participate (split arrive/wait) and *which* threads participate. In contrast, `this_thread_block.sync()` or `__syncthreads()` is applicable to the whole thread-block and `__syncwarp(mask)` to a specified subset of a warp. Nonetheless, if the intention of the user is to synchronize a full thread block or a full warp, we recommend using `__syncthreads()` and `__syncwarp()` respectively for better performance.

4.9.2. A Barrier's Phase: Arrival, Countdown, Completion, and Reset

An asynchronous barrier counts down from the expected arrival count to zero as participating threads call `bar.arrive()`. When the countdown reaches zero, the barrier is complete for the current phase. When the last call to `bar.arrive()` causes the countdown to reach zero, the countdown is automatically and atomically reset. The reset assigns the countdown to the expected arrival count, and moves the barrier to the next phase.

A token object of class `cuda::barrier::arrival_token`, as returned from `token=bar.arrive()`, is associated with the current phase of the barrier. A call to `bar.wait(std::move(token))` blocks the calling thread while the barrier is in the current phase, i.e., while the phase associated with the token matches the phase of the barrier. If the phase is advanced (because the countdown reaches

zero) before the call to `bar.wait(std::move(token))` then the thread does not block; if the phase is advanced while the thread is blocked in `bar.wait(std::move(token))`, the thread is unblocked.

It is essential to know when a reset could or could not occur, especially in non-trivial arrive/wait synchronization patterns.

- ▶ A thread's calls to `token=bar.arrive()` and `bar.wait(std::move(token))` must be sequenced such that `token=bar.arrive()` occurs during the barrier's current phase, and `bar.wait(std::move(token))` occurs during the same or next phase.
- ▶ A thread's call to `bar.arrive()` must occur when the barrier's counter is non-zero. After barrier initialization, if a thread's call to `bar.arrive()` causes the countdown to reach zero then a call to `bar.wait(std::move(token))` must happen before the barrier can be reused for a subsequent call to `bar.arrive()`.
- ▶ `bar.wait()` must only be called using a token object of the current phase or the immediately preceding phase. For any other values of the token object, the behavior is undefined.

For simple arrive/wait synchronization patterns, compliance with these usage rules is straightforward.

4.9.2.1 Warp Entanglement

Warp-divergence affects the number of times an arrive on operation updates the barrier. If the invoking warp is fully converged, then the barrier is updated once. If the invoking warp is fully diverged, then 32 individual updates are applied to the barrier.

Note

It is recommended that `arrive-on(bar)` invocations are used by converged threads to minimize updates to the barrier object. When code preceding these operations diverges threads, then the warp should be re-converged, via `__syncwarp` before invoking arrive-on operations.

4.9.3. Explicit Phase Tracking

An asynchronous barrier can have multiple phases depending on how many times it is used to synchronize threads and memory operations. Instead of using tokens to track barrier phase flips, we can directly track a phase using the `mbarrier_try_wait_parity()` family of functions available through the `cuda::ptx` and primitives APIs.

In its simplest form, the `cuda::ptx::mbarrier_try_wait_parity(uint64_t* bar, const uint32_t& phaseParity)` function waits for a phase with a particular parity. The `phaseParity` operand is the integer parity of either the current phase or the immediately preceding phase of the barrier object. An even phase has integer parity 0 and an odd phase has integer parity 1. When we initialize a barrier, its phase has parity 0. So the valid values of `phaseParity` are 0 and 1. Explicit phase tracking can be useful when tracking *asynchronous memory operations*, as it allows only a single thread to arrive on the barrier and set the transaction count, while other threads only wait for a parity-based phase flip. This can be more efficient than having all threads arrive on the barrier and use tokens. This functionality is only available for shared-memory barriers at thread-block and cluster scope.

CUDA C++ `cuda::barrier`

```

#include <cuda/ptx>
#include <cooperative_groups.h>

__device__ void compute(float *data, int iteration);

__global__ void split_arrive_wait(int iteration_count, float *data)
{
    using barrier_t = cuda::barrier<cuda::thread_scope_block>;
    __shared__ barrier_t bar;
    int parity = 0; // Initial phase parity is 0.
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        // Initialize barrier with expected arrival count.
        init(&bar, block.size());
    }
    block.sync();

    for (int i = 0; i < iteration_count; ++i)
    {
        /* code before arrive */

        // This thread arrives. Arrival does not block a thread.
        // Get a handle to the native barrier to use with cuda::ptx API.
        (void)cuda::ptx::mbarrier_arrive(cuda::device::barrier_native_
        ↪handle(bar));

        compute(data, i);

        // Wait for all threads participating in the barrier to complete mbarrier_
        ↪arrive().
        // Get a handle to the native barrier to use with cuda::ptx API.
        while (!cuda::ptx::mbarrier_try_wait_parity(cuda::device::barrier_
        ↪native_handle(bar), parity)) {}
        // Flip parity.
        parity ^= 1;

        /* code after wait */
    }
}

```

CUDA C++ `cuda::ptx`

```

#include <cuda/ptx>
#include <cooperative_groups.h>

__device__ void compute(float *data, int iteration);

__global__ void split_arrive_wait(int iteration_count, float *data)
{
    __shared__ uint64_t bar;
    int parity = 0; // Initial phase parity is 0.
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        // Initialize barrier with expected arrival count.
        cuda::ptx::mbarrier_init(&bar, block.size());
    }
    block.sync();

    for (int i = 0; i < iteration_count; ++i)
    {
        /* code before arrive */

        // This thread arrives. Arrival does not block a thread.
        (void)cuda::ptx::mbarrier_arrive(&bar);

        compute(data, i);

        // Wait for all threads participating in the barrier to complete mbarrier_
        arrive().
        while (!cuda::ptx::mbarrier_try_wait_parity(&bar, parity)) {}
        // Flip parity.
        parity ^= 1;

        /* code after wait */
    }
}

```

CUDA C primitives

```
#include <cuda_aware_barrier_primitives.h>
#include <cooperative_groups.h>

__device__ void compute(float *data, int iteration);

__global__ void split_arrive_wait(int iteration_count, float *data)
{
    __shared__ __mbarrier_t bar;
    bool parity = false; // Initial phase parity is false.
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        // Initialize barrier with expected arrival count.
        __mbarrier_init(&bar, block.size());
    }
    block.sync();

    for (int i = 0; i < iteration_count; ++i)
    {
        /* code before arrive */

        // This thread arrives. Arrival does not block a thread.
        (void)__mbarrier_arrive(&bar);

        compute(data, i);

        // Wait for all threads participating in the barrier to complete __
        ↪__mbarrier_arrive().
        while(!__mbarrier_try_wait_parity(&bar, parity, 1000)) {}
        parity ^= 1;

        /* code after wait */
    }
}
```

4.9.4. Early Exit

When a thread that is participating in a sequence of synchronizations must exit early from that sequence, that thread must explicitly drop out of participation before exiting. The remaining participating threads can proceed normally with subsequent arrive and wait operations.

CUDA C++ `cuda::barrier`

```

#include <cuda/barrier>
#include <cooperative_groups.h>

__device__ bool condition_check();

__global__ void early_exit_kernel(int N)
{
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        init(&bar, block.size());
    }
    block.sync();

    for (int i = 0; i < N; ++i)
    {
        if (condition_check())
        {
            bar.arrive_and_drop();
            return;
        }
        // Other threads can proceed normally.
        auto token = bar.arrive();

        /* code between arrive and wait */

        // Wait for all threads to arrive.
        bar.wait(std::move(token));

        /* code after wait */
    }
}

```

CUDA C primitives

```
#include <cuda_awbarrier_primitives.h>
#include <cooperative_groups.h>

__device__ bool condition_check();

__global__ void early_exit_kernel(int N)
{
    __shared__ __mbarrier_t bar;
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        __mbarrier_init(&bar, block.size());
    }
    block.sync();

    for (int i = 0; i < N; ++i)
    {
        if (condition_check())
        {
            __mbarrier_token_t token = __mbarrier_arrive_and_drop(&bar);
            return;
        }
        // Other threads can proceed normally.
        __mbarrier_token_t token = __mbarrier_arrive(&bar);

        /* code between arrive and wait */

        // Wait for all threads to arrive.
        while (!__mbarrier_try_wait(&bar, token, 1000)) {}

        /* code after wait */
    }
}
```

The `bar.arrive_and_drop()` operation arrives on the barrier to fulfill the participating thread's obligation to arrive in the **current** phase, and then decrements the expected arrival count for the **next** phase so that this thread is no longer expected to arrive on the barrier.

4.9.5. Completion Function

The `cuda::barrier` API supports an optional completion function. A `CompletionFunction` of `cuda::barrier<Scope, CompletionFunction>` is executed once per phase, after the last thread arrives and before any thread is unblocked from the wait. Memory operations performed by the threads that arrived at the barrier during the phase are visible to the thread executing the `CompletionFunction`, and all memory operations performed within the `CompletionFunction` are visible to all threads waiting at the barrier once they are unblocked from the wait.

CUDA C++ `cuda::barrier`

```

#include <cuda/barrier>
#include <cooperative_groups.h>
#include <functional>
namespace cg = cooperative_groups;

__device__ int divergent_compute(int *, int);
__device__ int independent_computation(int *, int);

__global__ void psum(int *data, int n, int *acc)
{
    auto block = cg::this_thread_block();

    constexpr int BlockSize = 128;
    __shared__ int smem[BlockSize];
    assert(BlockSize == block.size());
    assert(n % BlockSize == 0);

    auto completion_fn = [&]
    {
        int sum = 0;
        for (int i = 0; i < BlockSize; ++i)
        {
            sum += smem[i];
        }
        *acc += sum;
    };

    /* Barrier storage.
       Note: the barrier is not default-constructible because
       completion_fn is not default-constructible due
       to the capture. */
    using completion_fn_t = decltype(completion_fn);
    using barrier_t = cuda::barrier<cuda::thread_scope_block,
                                    completion_fn_t>;
    __shared__ std::aligned_storage<sizeof(barrier_t),
                                    alignof(barrier_t)>
        bar_storage;

    // Initialize barrier.
    barrier_t *bar = (barrier_t *)&bar_storage;
    if (block.thread_rank() == 0)
    {
        assert(*acc == 0);
        assert(blockDim.x == blockDim.y == blockDim.z == 1);
        new (bar) barrier_t{block.size(), completion_fn};
        /* equivalent to: init(bar, block.size(), completion_fn); */
    }
    block.sync();

    // Main loop.
    for (int i = 0; i < n; i += block.size())
    {
314 smem[block.thread_rank()] = data[i] + *acc;
        auto token = bar->arrive();
        // We can do independent computation here.
        bar->wait(std::move(token));
    }
}

```


4.9.6. Tracking Asynchronous Memory Operations

Asynchronous barriers can be used to track *asynchronous memory copies*. When an asynchronous copy operation is bound to a barrier, the copy operation automatically increments the expected count of the current barrier phase upon initiation and decrements it upon completion. This mechanism ensures that the barrier's `wait()` operation will block until all associated asynchronous memory copies have completed, providing a convenient way to synchronize multiple concurrent memory operations.

Starting with compute capability 9.0, asynchronous barriers in shared memory with thread-block or cluster scope can **explicitly** track asynchronous memory operations. We refer to these barriers as *asynchronous transaction barriers*. In addition to the expected arrival count, a barrier object can accept a **transaction count**, which can be used for tracking the completion of asynchronous transactions. The transaction count tracks the number of asynchronous transactions that are outstanding and yet to be complete, in units specified by the asynchronous memory operation (typically bytes). The transaction count to be tracked by the current phase can be set on arrival with `cuda::device::barrier_arrive_tx()` or directly with `cuda::device::barrier_expect_tx()`. When a barrier uses a transaction count, it blocks threads at the wait operation until all the producer threads have performed an arrive *and* the sum of all the transaction counts reaches an expected value.

CUDA C++ `cuda::barrier`

```
#include <cuda/barrier>
#include <cooperative_groups.h>

__global__ void track_kernel()
{
    __shared__ cuda::barrier<cuda::thread_scope_block> bar;
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        init(&bar, block.size());
    }
    block.sync();

    auto token = cuda::device::barrier_arrive_tx(bar, 1, 0);

    bar.wait(cuda::std::move(token));
}
```

CUDA C++ `cuda::ptx`

```

#include <cuda/ptx>
#include <cooperative_groups.h>

__global__ void track_kernel()
{
    __shared__ uint64_t bar;
    auto block = cooperative_groups::this_thread_block();

    if (block.thread_rank() == 0)
    {
        cuda::ptx::mbarrier_init(&bar, block.size());
    }
    block.sync();

    uint64_t token = cuda::ptx::mbarrier_arrive_expect_tx(cuda::ptx::sem_
→release, cuda::ptx::scope_cluster, cuda::ptx::space_shared, &bar, 1, 0);

    while (!cuda::ptx::mbarrier_try_wait(&bar, token)) {}
}

```

In this example, the `cuda::device::barrier_arrive_tx()` operation constructs an arrival token object associated with the phase synchronization point for the current phase. Then, decrements the arrival count by 1 and increments the expected transaction count by 0. Since the transaction count update is 0, the barrier is not tracking any transactions. The subsequent section on [Using the Tensor Memory Accelerator \(TMA\)](#) includes examples of tracking asynchronous memory operations.

4.9.7. Producer-Consumer Pattern Using Barriers

A thread block can be spatially partitioned to allow different threads to perform independent operations. This is most commonly done by assigning threads from different warps within the thread block to specific tasks. This technique is referred to as *warp specialization*.

This section shows an example of spatial partitioning in a producer-consumer pattern, where one subset of threads produces data that is concurrently consumed by the other (disjoint) subset of threads. A producer-consumer spatial partitioning pattern requires two one-sided synchronizations to manage a data buffer between the producer and consumer.

Producer	Consumer
wait for buffer to be ready to be filled	signal buffer is ready to be filled
produce data and fill the buffer	
signal buffer is filled	wait for buffer to be filled
	consume data in filled buffer

Producer threads wait for consumer threads to signal that the buffer is ready to be filled; however, consumer threads do not wait for this signal. Consumer threads wait for producer threads to signal that

the buffer is filled; however, producer threads do not wait for this signal. For full producer/consumer concurrency this pattern has (at least) double buffering where each buffer requires two barriers.

CUDA C++ `cuda::barrier`

```

#include <cuda/barrier>

using barrier_t = cuda::barrier<cuda::thread_scope_block>;

__device__ void produce(barrier_t ready[], barrier_t filled[], float
    ↪ *buffer, int buffer_len, float *in, int N)
{
    for (int i = 0; i < N / buffer_len; ++i)
    {
        ready[i % 2].arrive_and_wait(); /* wait for buffer_(i%2) to be ready to
    ↪ be filled */
        /* produce, i.e., fill in, buffer_(i%2) */
        barrier_t::arrival_token token = filled[i % 2].arrive(); /* buffer_(i
    ↪ %2) is filled */
    }
}

__device__ void consume(barrier_t ready[], barrier_t filled[], float
    ↪ *buffer, int buffer_len, float *out, int N)
{
    barrier_t::arrival_token token1 = ready[0].arrive(); /* buffer_0 is ready
    ↪ for initial fill */
    barrier_t::arrival_token token2 = ready[1].arrive(); /* buffer_1 is ready
    ↪ for initial fill */
    for (int i = 0; i < N / buffer_len; ++i)
    {
        filled[i % 2].arrive_and_wait(); /* wait for buffer_(i%2) to be filled */
        /* consume buffer_(i%2) */
        barrier_t::arrival_token token3 = ready[i % 2].arrive(); /* buffer_(i
    ↪ %2) is ready to be re-filled */
    }
}

__global__ void producer_consumer_pattern(int N, float *in, float *out, int
    ↪ buffer_len)
{
    constexpr int warpSize = 32;

    /* Shared memory buffer declared below is of size 2 * buffer_len
       so that we can alternatively work between two buffers.
       buffer_0 = buffer and buffer_1 = buffer + buffer_len */
    __shared__ extern float buffer[];

    /* bar[0] and bar[1] track if buffers buffer_0 and buffer_1 are ready to be
    ↪ filled,
       while bar[2] and bar[3] track if buffers buffer_0 and buffer_1 are
    ↪ filled-in respectively */
    #pragma nv_diag_suppress static_var_with_dynamic_init
    __shared__ barrier_t bar[4];

    if (threadIdx.x < 4)
    {
318 init(bar + threadIdx.x, blockDim.x);
    }
    __syncthreads();
}

```


CUDA C++ `cuda::ptx`

```

#include <cuda/ptx>

__device__ void produce(barrier ready[], barrier filled[], float *buffer,
    ↪ int buffer_len, float *in, int N)
{
    for (int i = 0; i < N / buffer_len; ++i)
    {
        uint64_t token1 = cuda::ptx::mbarrier_arrive(ready[i % 2]);
        while(!cuda::ptx::mbarrier_try_wait(&ready[i % 2], token1)) {} /* wait
    ↪ for buffer_(i%2) to be ready to be filled */
        /* produce, i.e., fill in, buffer_(i%2) */
        uint64_t token2 = cuda::ptx::mbarrier_arrive(&filled[i % 2]); /* buffer_
    ↪ (i%2) is filled */
    }
}

__device__ void consume(barrier ready[], barrier filled[], float *buffer,
    ↪ buffer_len, float *out, int N)
{
    uint64_t token1 = cuda::ptx::mbarrier_arrive(&ready[0]); /* buffer_0 is
    ↪ ready for initial fill */
    uint64_t token2 = cuda::ptx::mbarrier_arrive(&ready[1]); /* buffer_1 is
    ↪ ready for initial fill */
    for (int i = 0; i < N / buffer_len; ++i)
    {
        uint64_t token3 = cuda::ptx::mbarrier_arrive(&filled[i % 2]);
        while(!cuda::ptx::mbarrier_try_wait(&filled[i % 2], token3x)) {} /*
    ↪ wait for buffer_(i%2) to be filled */
        /* consume buffer_(i%2) */
        uint64_t token4 = cuda::ptx::mbarrier_arrive(&ready[i % 2]); /* buffer_
    ↪ (i%2) is ready to be re-filled */
    }
}

__global__ void producer_consumer_pattern(int N, float *in, float *out, int
    ↪ buffer_len)
{
    constexpr int warpSize = 32;

    /* Shared memory buffer declared below is of size 2 * buffer_len
       so that we can alternatively work between two buffers.
       buffer_0 = buffer and buffer_1 = buffer + buffer_len */
    __shared__ extern float buffer[];

    /* bar[0] and bar[1] track if buffers buffer_0 and buffer_1 are ready to be
    ↪ filled,
       while bar[2] and bar[3] track if buffers buffer_0 and buffer_1 are
    ↪ filled-in respectively */
    #pragma nv_diag_suppress static_var_with_dynamic_init
    __shared__ uint64_t bar[4];

    if (threadIdx.x < 4)
320{
    cuda::ptx::mbarrier_init(bar + block.thread_rank(), block.size());
    }
    __syncthreads();
}

```


CUDA C primitives

```

#include <cuda_aware_barrier_primitives.h>

__device__ void produce(__mbarrier_t ready[], __mbarrier_t filled[], float
    ↪ *buffer, int buffer_len, float *in, int N)
{
    for (int i = 0; i < N / buffer_len; ++i)
    {
        __mbarrier_token_t token1 = __mbarrier_arrive(&ready[i % 2]); /* wait
    ↪ for buffer_(i%2) to be ready to be filled */
        while(!__mbarrier_try_wait(&ready[i % 2], token1, 1000)) {}
        /* produce, i.e., fill in, buffer_(i%2) */
        __mbarrier_token_t token2 = __mbarrier_arrive(filled[i % 2]); /*
    ↪ buffer_(i%2) is filled */
    }
}

__device__ void consume(__mbarrier_t ready[], __mbarrier_t filled[], float
    ↪ *buffer, int buffer_len, float *out, int N)
{
    __mbarrier_token_t token1 = __mbarrier_arrive(&ready[0]); /* buffer_0 is
    ↪ ready for initial fill */
    __mbarrier_token_t token2 = __mbarrier_arrive(&ready[1]); /* buffer_1 is
    ↪ ready for initial fill */
    for (int i = 0; i < N / buffer_len; ++i)
    {
        __mbarrier_token_t token3 = __mbarrier_arrive(&filled[i % 2]);
        while(!__mbarrier_try_wait(&filled[i % 2], token3, 1000)) {}
        /* consume buffer_(i%2) */
        __mbarrier_token_t token4 = __mbarrier_arrive(&ready[i % 2]); /* buffer_
    ↪ (i%2) is ready to be re-filled */
    }
}

__global__ void producer_consumer_pattern(int N, float *in, float *out, int
    ↪ buffer_len)
{
    constexpr int warpSize = 32;

    /* Shared memory buffer declared below is of size 2 * buffer_len
    so that we can alternatively work between two buffers.
    buffer_0 = buffer and buffer_1 = buffer + buffer_len */
    __shared__ extern float buffer[];

    /* bar[0] and bar[1] track if buffers buffer_0 and buffer_1 are ready to be
    ↪ filled,
    while bar[2] and bar[3] track if buffers buffer_0 and buffer_1 are
    ↪ filled-in respectively */
    #pragma nv_diag_suppress static_var_with_dynamic_init
    __shared__ __mbarrier_t bar[4];

    if (threadIdx.x < 4)
    {
322 __mbarrier_init(bar + threadIdx.x, blockDim.x);
    }
    __syncthreads();
}

```


In this example, the first warp is specialized as the producer and the remaining warps are specialized as consumers. All producer and consumer threads participate (call `bar.arrive()` or `bar.arrive_and_wait()`) in each of the four barriers so the expected arrival counts are equal to `block.size()`.

A producer thread waits for the consumer threads to signal that the shared memory buffer can be filled. In order to wait for a barrier, a producer thread must first arrive on that `ready[i%2].arrive()` to get a token and then `ready[i%2].wait(token)` with that token. For simplicity, `ready[i%2].arrive_and_wait()` combines these operations.

```
bar.arrive_and_wait();
/* is equivalent to */
bar.wait(bar.arrive());
```

Producer threads compute and fill the ready buffer, they then signal that the buffer is filled by arriving on the filled barrier, `filled[i%2].arrive()`. A producer thread does not wait at this point, instead it waits until the next iteration's buffer (double buffering) is ready to be filled.

A consumer thread begins by signaling that both buffers are ready to be filled. A consumer thread does not wait at this point, instead it waits for this iteration's buffer to be filled, `filled[i%2].arrive_and_wait()`. After the consumer threads consume the buffer they signal that the buffer is ready to be filled again, `ready[i%2].arrive()`, and then wait for the next iteration's buffer to be filled.

4.10. Pipelines

Pipelines, introduced in [Advanced Synchronization Primitives](#), are a mechanism for staging work and coordinating multi-buffer producer-consumer patterns, commonly used to overlap compute with [asynchronous data copies](#).

This section provides details on how to use pipelines mainly via the `cuda::pipeline` API (with pointers to primitives where applicable).

4.10.1. Initialization

A `cuda::pipeline` can be created at different thread scopes. For a scope other than `cuda::thread_scope_thread`, a `cuda::pipeline_shared_state<scope, count>` object is required to coordinate the participating threads. This state encapsulates the finite resources that allow a pipeline to process up to `count` concurrent stages.

```
// Create a pipeline at thread scope
constexpr auto scope = cuda::thread_scope_thread;
cuda::pipeline<scope> pipeline = cuda::make_pipeline();
```

```
// Create a pipeline at block scope
constexpr auto scope = cuda::thread_scope_block;
constexpr auto stages_count = 2;
__shared__ cuda::pipeline_shared_state<scope, stages_count> shared_state;
auto pipeline = cuda::make_pipeline(group, &shared_state);
```

Pipelines can be either *unified* or *partitioned*. In a unified pipeline, all the participating threads are both producers and consumers. In a partitioned pipeline, each participating thread is either a producer or a consumer and its role cannot change during the lifetime of the pipeline object. A thread-local

pipeline cannot be partitioned. To create a partitioned pipeline, we need to provide either the number of producers or the role of the thread to `cuda::make_pipeline()`.

```
// Create a partitioned pipeline at block scope where only thread 0 is a
// producer
constexpr auto scope = cuda::thread_scope_block;
constexpr auto stages_count = 2;
__shared__ cuda::pipeline_shared_state<scope, stages_count> shared_state;
auto thread_role = (group.thread_rank() == 0) ? cuda::pipeline_role::producer
    : cuda::pipeline_role::consumer;
auto pipeline = cuda::make_pipeline(group, &shared_state, thread_role);
```

To support partitioning, a shared `cuda::pipeline` incurs additional overheads, including using a set of shared memory barriers per stage for synchronization. These are used even when the pipeline is unified and could use `__syncthreads()` instead. Thus, it is preferable to use thread-local pipelines which avoid these overheads when possible.

4.10.2. Submitting Work

Committing work to a pipeline stage involves:

- ▶ Collectively *acquiring* the pipeline *head* from a set of producer threads using `pipeline.producer_acquire()`.
- ▶ Submitting asynchronous operations, e.g., `memcpy_async`, to the pipeline head.
- ▶ Collectively *committing* (advancing) the pipeline head using `pipeline.producer_commit()`.

If all resources are in use, `pipeline.producer_acquire()` blocks producer threads until the resources of the next pipeline stage are released by consumer threads.

4.10.3. Consuming Work

Consuming work from a previously committed stage involves:

- ▶ Collectively waiting for the stage to complete, e.g., using `pipeline.consumer_wait()` to wait on the tail (oldest) stage, from a set of consumer threads.
- ▶ Collectively *releasing* the stage using `pipeline.consumer_release()`.

With `cuda::pipeline<cuda::thread_scope_thread>` one can also use the `cuda::pipeline_consumer_wait_prior<N>()` friend function to wait for all except the last `N` stages to complete, similar to `__pipeline_wait_prior(N)` in the primitives API.

4.10.4. Warp Entanglement

The pipeline mechanism is shared among CUDA threads in the same warp. This sharing causes sequences of submitted operations to be entangled within a warp, which can impact performance under certain circumstances.

Commit. The commit operation is coalesced such that the pipeline's sequence is incremented once for all converged threads that invoke the commit operation and their submitted operations are batched together. If the warp is fully converged, the sequence is incremented by one and all submitted operations will be batched in the same stage of the pipeline; if the warp is fully diverged, the sequence is incremented by 32 and all submitted operations will be spread to different stages.

- Let PB be the warp-shared pipeline's *actual* sequence of operations.

$PB = \{BP0, BP1, BP2, \dots, BPL\}$

- Let TB be a thread's *perceived* sequence of operations, as if the sequence were only incremented by this thread's invocation of the commit operation.

$TB = \{BT0, BT1, BT2, \dots, BTL\}$

The `pipeline::producer_commit()` return value is from the thread's *perceived* batch sequence.

- An index in a thread's perceived sequence always aligns to an equal or larger index in the actual warp-shared sequence. The sequences are equal only when all commit operations are invoked from fully converged threads.

$BT_n \equiv BP_m$ where $n \leq m$

For example, when a warp is fully diverged:

- The warp-shared pipeline's actual sequence would be: $PB = \{0, 1, 2, 3, \dots, 31\}$ ($PL=31$).
- The perceived sequence for each thread of this warp would be:
 - Thread 0: $TB = \{0\}$ ($TL=0$)
 - Thread 1: $TB = \{0\}$ ($TL=0$)
 - ...
 - Thread 31: $TB = \{0\}$ ($TL=0$)

Wait. A CUDA thread invokes `pipeline::consumer_wait()` or `pipeline_consumer_wait_prior<N>()` to wait for batches in the *perceived* sequence TB to complete. Note that `pipeline::consumer_wait()` is equivalent to `pipeline_consumer_wait_prior<N>()`, where $N = PL$.

The *wait prior* variants wait for batches in the *actual* sequence at least up to and including $PL - N$. Since $TL \leq PL$, waiting for batch up to and including $PL - N$ includes waiting for batch $TL - N$. Thus, when $TL < PL$, the thread will unintentionally wait for additional, more recent batches. In the extreme fully-diverged warp example above, each thread could wait for all 32 batches.

Note

It is recommended that commit invocations are by converged threads to not over-wait, by keeping threads' perceived sequence of batches aligned with the actual sequence.

When code preceding these operations diverges threads, then the warp should be re-converged, via `__syncwarp` before invoking commit operations.

4.10.5. Early Exit

When a thread that is participating in a pipeline must exit early, that thread must explicitly drop out of participation before exiting using `cuda::pipeline::quit()`. The remaining participating threads can proceed normally with subsequent operations.

4.10.6. Tracking Asynchronous Memory Operations

The following example demonstrates how to collectively copy data from global to shared memory with asynchronous memory copies using a pipeline to keep track of the copy operations. Each thread uses its own pipeline to independently submit memory copies and then wait for them to complete and consume the data. For more details on asynchronous data copies, see [Section 3.2.5](#).

CUDA C++ `cuda::pipeline`

```

#include <cuda/pipeline>

__global__ void example_kernel(const float *in)
{
    constexpr int block_size = 128;
    __shared__ __align__(sizeof(float)) float buffer[4 * block_size];

    // Create a unified pipeline per thread
    cuda::pipeline<cuda::thread_scope_thread> pipeline = cuda::make_
    ↪ pipeline();

    // First stage of memory copies
    pipeline.producer_acquire();
    // Every thread fetches one element of the first block
    cuda::memcpy_async(buffer, in, sizeof(float), pipeline);
    pipeline.producer_commit();

    // Second stage of memory copies
    pipeline.producer_acquire();
    // Every thread fetches one element of the second and third block
    cuda::memcpy_async(buffer + block_size, in + block_size, sizeof(float),
    ↪ pipeline);
    cuda::memcpy_async(buffer + 2 * block_size, in + 2 * block_size,
    ↪ sizeof(float), pipeline);
    pipeline.producer_commit();

    // Third stage of memory copies
    pipeline.producer_acquire();
    // Every thread fetches one element of the last block
    cuda::memcpy_async(buffer + 3 * block_size, in + 3 * block_size,
    ↪ sizeof(float), pipeline);
    pipeline.producer_commit();

    // Wait for the oldest stage (waits for first stage)
    pipeline.consumer_wait();
    pipeline.consumer_release();

    // __syncthreads();
    // Use data from the first stage

    // Wait for the oldest stage (waits for second stage)
    pipeline.consumer_wait();
    pipeline.consumer_release();

    // __syncthreads();
    // Use data from the second stage

    // Wait for the oldest stage (waits for third stage)
    pipeline.consumer_wait();
    pipeline.consumer_release();

    // __syncthreads();
    // Use data from the third stage
}

```

CUDA C primitives

```

#include <cuda_pipeline.h>

__global__ void example_kernel(const float *in)
{
    constexpr int block_size = 128;
    __shared__ __align__(sizeof(float)) float buffer[4 * block_size];

    // First batch of memory copies
    // Every thread fetches one element of the first block
    __pipeline_memcpy_async(buffer, in, sizeof(float));
    __pipeline_commit();

    // Second batch of memory copies
    // Every thread fetches one element of the second and third block
    __pipeline_memcpy_async(buffer + block_size, in + block_size,
↪ sizeof(float));
    __pipeline_memcpy_async(buffer + 2 * block_size, in + 2 * block_size,
↪ sizeof(float));
    __pipeline_commit();

    // Third batch of memory copies
    // Every thread fetches one element of the last block
    __pipeline_memcpy_async(buffer + 3 * block_size, in + 3 * block_size,
↪ sizeof(float));
    __pipeline_commit();

    // Wait for all except the last two batches of memory copies (waits for
↪ first batch)
    __pipeline_wait_prior(2);

    // __syncthreads();
    // Use data from the first batch

    // Wait for all except the last batch of memory copies (waits for second
↪ batch)
    __pipeline_wait_prior(1);

    // __syncthreads();
    // Use data from the second batch

    // Wait for all batches of memory copies (waits for third batch)
    __pipeline_wait_prior(0);

    // __syncthreads();
    // Use data from the last batch
}

```

4.10.7. Producer-Consumer Pattern using Pipelines

In [Section 4.9.7](#), we showed how a thread block can be spatially partitioned to implement a producer-consumer pattern using *asynchronous barriers*. With `cuda::pipeline`, this can be simplified using a single partitioned pipeline with one stage per data buffer instead of two asynchronous barriers per buffer.

CUDA C++ `cuda::pipeline`

```

#include <cuda/pipeline>
#include <cooperative_groups.h>

#pragma nv_diag_suppress static_var_with_dynamic_init

using pipeline = cuda::pipeline<cuda::thread_scope_block>;

__device__ void produce(pipeline &pipe, int num_stages, int stage, int num_
→batches, int batch, float *buffer, int buffer_len, float *in, int N)
{
    if (batch < num_batches)
    {
        pipe.producer_acquire();
        /* copy data from in(batch) to buffer(stage) using asynchronous memory
→copies */
        pipe.producer_commit();
    }
}

__device__ void consume(pipeline &pipe, int num_stages, int stage, int num_
→batches, int batch, float *buffer, int buffer_len, float *out, int N)
{
    pipe.consumer_wait();
    /* consume buffer(stage) and update out(batch) */
    pipe.consumer_release();
}

__global__ void producer_consumer_pattern(float *in, float *out, int N, int
→buffer_len)
{
    auto block = cooperative_groups::this_thread_block();

    /* Shared memory buffer declared below is of size 2 * buffer_len
    so that we can alternatively work between two buffers.
    buffer_0 = buffer and buffer_1 = buffer + buffer_len */
    __shared__ extern float buffer[];

    const int num_batches = N / buffer_len;

    /* Create a partitioned pipeline with 2 stages where half the threads are
→producers and the other half are consumers.
    constexpr auto scope = cuda::thread_scope_block;
    constexpr int num_stages = 2;
    cuda::std::size_t producer_count = block.size() / 2;
    __shared__ cuda::pipeline_shared_state<scope, num_stages> shared_state;
    pipeline pipe = cuda::make_pipeline(block, &shared_state, producer_count);

    // Fill the pipeline
    if (block.thread_rank() < producer_count)
    {
        for (int s = 0; s < num_stages; ++s)
        {
            produce(pipe, num_stages, s, num_batches, s, buffer, buffer_len, in,
→N);
        }
    }
}

```


In this example, we use half of the threads in the thread block as producers and the other half as consumers. As a first step, we need to create a `cuda::pipeline` object. Since we want some threads to be producers and some to be consumers, we need to use a **partitioned** pipeline with `cuda::thread_scope_block`. Partitioned pipelines require a `cuda::pipeline_shared_state` to coordinate the participating threads. We initialize the state for a 2-stage pipeline in thread-block scope and then call `cuda::make_pipeline()`. Next, producer threads fill the pipeline by submitting asynchronous copies from `in` to `buffer`. At this point all data copies are in-flight. Finally, in the main loop, we go over all of the batches of data and depending on whether a thread is a producer or consumer, we either submit another asynchronous copy for a future batch or consume the current batch.

4.11. Asynchronous Data Copies

Building on [Section 3.2.5](#), this section provides detailed guidance and examples for asynchronous data movement within the GPU memory hierarchy. It covers LDGSTS for element-wise copies, the Tensor Memory Accelerator (TMA) for bulk (one-dimensional and multi-dimensional) transfers, and STAS for register to distributed shared memory copies, and shows how these mechanisms integrate with *asynchronous barriers* and *pipelines*.

4.11.1. Using LDGSTS

Many CUDA applications require frequent data movement between global and shared memory. Often, this involves copying smaller data elements or performing irregular memory access patterns. The primary goal of LDGSTS (CC 8.0+, see [PTX documentation](#)) is to provide an efficient asynchronous data transfer mechanism from global memory to shared memory for smaller, element-wise data transfers while enabling better utilization of compute resources through overlapped execution.

Dimensions. LDGSTS supports copying 4, 8, or 16 bytes. Copying 4 or 8 bytes always happens in the so called L1 ACCESS mode, in which case data is also cached in the L1, while copying 16-bytes enables the L1 BYPASS mode, in which case the L1 is not polluted.

Source and destination. The only direction supported for asynchronous copy operations with LDGSTS is from global to shared memory. The pointers need to be aligned to 4, 8, or 16 bytes depending on the size of the data being copied. Best performance is achieved when the alignment of both shared memory and global memory is 128 bytes.

Asynchronicity. Data transfers using LDGSTS are *asynchronous* and are modeled as async thread operations (see *Async Thread and Async Proxy*). This allows the initiating thread to continue computing while the hardware asynchronously copies the data. *Whether the data transfer occurs asynchronously in practice is up to the hardware implementation and may change in the future.*

LDGSTS must provide a signal when the operation is complete. LDGSTS can use *shared memory barriers* or *pipelines* as mechanisms to provide completion signals. By default, each thread only waits for its own LDGSTS copies. Thus, if you use LDGSTS to prefetch some data that will be shared with other threads, a `__syncthreads()` is necessary after synchronizing with the LDGSTS completion mechanism.

Table 18: Asynchronous copies with possible source and destination memory spaces and completion mechanisms using LDGSTS. An empty cell indicates that a source-destination pair is not supported.

Direction		Asynchronous Copy (LDGSTS, CC 8.0+)		
Source	Destination	Completion nism	Mecha-	API
global	global			
shared::cta	global			
global	shared::cta	shared memory barrier, pipeline		<code>cuda::memcpy_async</code> , <code>cooperative_groups::memcpy_async</code> , <code>__pipeline_memcpy_async</code>
global	shared::cluster			
shared::cluster	shared::cta			
shared::cta	shared::cta			

In the following sections, we will demonstrate how to use LDGSTS through examples and explain the differences between the different APIs.

4.11.1.1 Batching Loads in Conditional Code

In this stencil example, the first warp of the thread block is responsible for collectively loading all the required data from the center as well as the left and right halos. With synchronous copies, due to the conditional nature of the code, the compiler may choose to generate a sequence of load-from-global (LDG) store-to-shared (STS) instructions instead of 3 LDGs followed by 3 STSs, which would be the optimal way to load the data to hide the global memory latency.

```
__global__ void stencil_kernel(const float *left, const float *center, const
    float *right)
{
    // Left halo (8 elements) - center (32 elements) - right halo (8 elements)
    __shared__ float buffer[8 + 32 + 8];
    const int tid = threadIdx.x;

    if (tid < 8) {
        buffer[tid] = left[tid]; // Left halo
    } else if (tid >= 32 - 8) {
        buffer[tid + 16] = right[tid]; // Right halo
    }
    if (tid < 32) {
        buffer[tid + 8] = center[tid]; // Center
    }
    __syncthreads();
}
```

(continues on next page)

(continued from previous page)

```
// Compute stencil  
}
```

To ensure that the data is loaded in the optimal way, we can replace the synchronous memory copies with asynchronous copies that load data directly from global memory to shared memory. This not only reduces register usage by copying the data directly to shared memory, but also ensures all loads from global memory are in-flight.

CUDA C++ `cuda::memcpy_async`

```

#include <cooperative_groups.h>
#include <cuda/barrier>

__global__ void stencil_kernel(const float *left, const float *center,
    ↪ const float *right)
{
    auto block = cooperative_groups::this_thread_block();
    auto thread = cooperative_groups::this_thread();
    using barrier_t = cuda::barrier<cuda::thread_scope_block>;
    __shared__ barrier_t barrier;
    __shared__ float buffer[8 + 32 + 8];

    // Initialize synchronization object.
    if (block.thread_rank() == 0) {
        init(&barrier, block.size());
    }
    __syncthreads();

    // Version 1: Issue the copies in individual threads.
    if (tid < 8) {
        cuda::memcpy_async(buffer + tid, left + tid, cuda::aligned_size_t<4>
    ↪ (sizeof(float)), barrier); // Left halo
        // or cuda::memcpy_async(thread, buffer + tid, left + tid,
    ↪ cuda::aligned_size_t<4>(sizeof(float)), barrier);
    } else if (tid >= 32 - 8) {
        cuda::memcpy_async(buffer + tid + 16, right + tid, cuda::aligned_
    ↪ size_t<4>(sizeof(float)), barrier); // Right halo
        // or cuda::memcpy_async(thread, buffer + tid + 16, right + tid,
    ↪ cuda::aligned_size_t<4>(sizeof(float)), barrier);
    }
    if (tid < 32) {
        cuda::memcpy_async(buffer + 40, right + tid, cuda::aligned_size_t<4>
    ↪ (sizeof(float)), barrier); // Center
        // or cuda::memcpy_async(thread, buffer + 40, right + tid,
    ↪ cuda::aligned_size_t<4>(sizeof(float)), barrier);
    }

    // Version 2: Cooperatively issue the copies across all threads.
    cuda::memcpy_async(block, buffer, left, cuda::aligned_size_t<4>(8 *
    ↪ sizeof(float)), barrier); // Left halo
    cuda::memcpy_async(block, buffer + 8, center, cuda::aligned_size_t<4>
    ↪ (32 * sizeof(float)), barrier); // Center
    cuda::memcpy_async(block, buffer + 40, right, cuda::aligned_size_t<4>(8
    ↪ * sizeof(float)), barrier); // Right halo

    // Wait for all copies to complete.
    barrier.arrive_and_wait();
    __syncthreads();

    // Compute stencil
}

```

CUDA C++ cooperative_groups::memcpy_async

```

#include <cooperative_groups.h>
#include <cooperative_groups/memcpy_async.h>

namespace cg = cooperative_groups;

__global__ void stencil_kernel(const float *left, const float *center,
    ↪ const float *right)
{
    cg::thread_block block = cg::this_thread_block();
    // Left halo (8 elements) - center (32 elements) - right halo (8
    ↪ elements).
    __shared__ float buffer[8 + 32 + 8];

    // Cooperatively issue the copies across all threads.
    cg::memcpy_async(block, buffer, left, 8 * sizeof(float)); // Left halo
    cg::memcpy_async(block, buffer + 8, center, 32 * sizeof(float)); //
    ↪ Center
    cg::memcpy_async(block, buffer + 40, right, 8 * sizeof(float)); //
    ↪ Right halo
    cg::wait(block); // Waits for all copies to complete.
    __syncthreads();

    // Compute stencil.
}

```

CUDA C primitives

```

#include <cuda_pipeline.h>

__global__ void stencil_kernel(const float *left, const float *center,
    ↪ const float *right)
{
    // Left halo (8 elements) - center (32 elements) - right halo (8
    ↪ elements).
    __shared__ float buffer[8 + 32 + 8];
    const int tid = threadIdx.x;

    if (tid < 8) {
        __pipeline_memcpy_async(buffer + tid, left + tid, sizeof(float)); //
    ↪ Left halo
    } else if (tid >= 32 - 8) {
        __pipeline_memcpy_async(buffer + tid + 16, right + tid,
    ↪ sizeof(float)); // Right halo
    }
    if (tid < 32) {
        __pipeline_memcpy_async(buffer + tid + 8, center + tid,
    ↪ sizeof(float)); // Center
    }
    __pipeline_commit();
    __pipeline_wait_prior(0);
    __syncthreads();

    // Compute stencil.
}

```

The `cuda::memcpy_async` overload for `cuda::barrier` enables synchronizing asynchronous data transfers using an *asynchronous barrier*. This overload executes the copy operation as-if performed by another thread bound to the barrier by incrementing the expected count of the current phase on creation, and decrementing it on completion of the copy operation, such that the phase of the barrier will only advance when all threads participating in the barrier have arrived, and all `memcpy_async` bound to the current phase of the barrier have completed. We use a block-wide barrier, where all threads in the block participate, and merge the arrival and wait on the barrier with `arrive_and_wait`, since we do not perform any work between the phases.

Note that we can either use thread-level copies (version 1) or collective copies (version 2) to achieve the same result. In version 2, the API will automatically handle how the copies are done under the hood. In both versions, we use `cuda::aligned_size_t<4>()` to inform the compiler that the data is aligned to 4 bytes and the size of the data to copy is a multiple of 4 to enable use of LDGSTS. Note that for interoperability with `cuda::barrier`, `cuda::memcpy_async` from the `cuda/barrier` header is used here.

The `cooperative_groups::memcpy_async` implementation coordinates the memory transfers collectively across all threads in the block, but synchronizes completion with `cg::wait(block)` instead of explicit barrier operations.

The implementation based on the low-level primitives uses `__pipeline_memcpy_async()` to initiate element-wise memory transfers, `__pipeline_commit()` to commit the batch of copies, and `__pipeline_wait_prior(0)` to wait for all operations in the pipeline to complete. This provides the

most direct control at the expense of more verbose code compared to the higher-level APIs. It also ensures LDGSTS will be used under the hood, which is not guaranteed with the higher-level APIs.

Note

The `cooperative_groups::memcpy_async` API is less efficient than the other APIs in this example because it automatically commits each copy operation immediately upon launch, preventing the optimization of batching multiple copies before a single commit operation that the other APIs enable.

4.11.1.2 Prefetching Data

In this example, we will demonstrate how to use asynchronous data copies to prefetch data from global memory to shared memory. In an iterative copy and compute pattern, this allows hiding the latency of data transfers of future iterations with computation on the current iteration, potentially increasing bytes-in-flight.

CUDA C++ `cuda::memcpy_async`

```
#include <cooperative_groups.h>
#include <cuda/pipeline>

template <size_t num_stages = 2 /* Pipeline with num_stages stages */>
__global__ void prefetch_kernel(int* global_out, int const* global_in, size_t
    size, size_t batch_size) {
    auto grid = cooperative_groups::this_grid();
    auto block = cooperative_groups::this_thread_block();
    auto thread = cooperative_groups::this_thread();
    assert(size == batch_size * grid.size()); // Assume input size fits
    batch_size * grid_size

    extern __shared__ int shared[]; // num_stages * block.size() *
    sizeof(int) bytes
    size_t shared_offset[num_stages];
    for (int s = 0; s < num_stages; ++s) shared_offset[s] = s * block.
    size();

    cuda::pipeline<cuda::thread_scope_thread> pipeline = cuda::make_
    pipeline();

    auto block_batch = [&](size_t batch) -> int {
        return block.group_index().x * block.size() + grid.size() * batch;
    };

    // Fill the pipeline with the first `num_stages` batches.
    for (int s = 0; s < num_stages; ++s) {
        pipeline.producer_acquire();
        cuda::memcpy_async(shared + shared_offset[s] + tid, global_in +
    block_batch(s) + tid, cuda::aligned_size_t<4>(sizeof(int)), pipeline);
        pipeline.producer_commit();
    }

    int stage = 0;

    // compute_batch: next batch to process
    // fetch_batch: next batch to fetch from global memory
    for (size_t compute_batch = 0, fetch_batch = num_stages; compute_batch
    < batch_size; ++compute_batch, ++fetch_batch) {
        // Wait for the first requested stage to complete.
        constexpr size_t pending_batches = num_stages - 1;
        cuda::pipeline_consumer_wait_prior<pending_batches>(pipeline);
        __syncthreads(); // Not required if each thread works on the data it
    copied.

        // Compute on the current batch
        compute(global_out + block_batch(compute_batch) + tid, shared +
    shared_offset[stage] + tid);

        // Release the current stage.
        pipeline.consumer_release();
        __syncthreads(); // Not required if each thread works on the data it
    copied.

        // Load future stage `num_stages` ahead of current compute batch.
        pipeline.producer_acquire();
```


CUDA C++ `cooperative_groups::memcpy_async`

```

#include <cooperative_groups.h>
#include <cooperative_groups/memcpy_async.h>

namespace cg = cooperative_groups;

template <size_t num_stages = 2 /* Pipeline with num_stages stages */>
__global__ void prefetch_kernel(int* global_out, int const* global_in, size_t
    size, size_t batch_size) {
    auto grid = cooperative_groups::this_grid();
    auto block = cooperative_groups::this_thread_block();
    assert(size == batch_size * grid.size()); // Assume input size fits
    batch_size * grid_size

    extern __shared__ int shared[]; // num_stages * block.size() *
    sizeof(int) bytes
    size_t shared_offset[num_stages];
    for (int s = 0; s < num_stages; ++s) shared_offset[s] = s * block.
    size();

    cuda::pipeline<cuda::thread_scope_thread> pipeline = cuda::make_
    pipeline();

    auto block_batch = [&](size_t batch) -> int {
        return block.group_index().x * block.size() + grid.size() * batch;
    };

    // Fill the pipeline with the first `num_stages` batches.
    for (int s = 0; s < num_stages; ++s) {
        size_t block_batch_idx = block_batch(s);
        cg::memcpy_async(block, shared + shared_offset[s], global_in +
    block_batch_idx, cuda::aligned_size_t<4>(sizeof(int)));
    }

    int stage = 0;

    // compute_batch: next batch to process
    // fetch_batch: next batch to fetch from global memory
    for (size_t compute_batch = 0, fetch_batch = num_stages; compute_batch
    < batch_size; ++compute_batch, ++fetch_batch) {
        // Wait for the first requested stage to complete.
        size_t pending_batches = (fetch_batch < batch_size - num_stages) ?
    num_stages - 1 : batch_size - fetch_batch - 1;
        cg::wait_prior(pending_batches);
        __syncthreads(); // Not required if each thread works on the data it
    copied.

        // Compute on the current batch.
        compute(global_out + block_batch(compute_batch) + tid, shared +
    shared_offset[stage] + tid);

        __syncthreads(); // Not required if each thread works on the data it
    copied.

        // Load future stage `num_stages` ahead of current compute batch.
        size_t fetch_batch_idx = block_batch(fetch_batch);
        if (fetch_batch < batch_size) {

```


CUDA C primitives

```

#include <cooperative_groups.h>
#include <cuda_aware_barrier_primitives.h>

template <size_t num_stages = 2 /* Pipeline with num_stages stages */>
__global__ void prefetch_kernel(int* global_out, int const* global_in, size_t
    size, size_t batch_size) {
    auto grid = cooperative_groups::this_grid();
    auto block = cooperative_groups::this_thread_block();
    assert(size == batch_size * grid.size()); // Assume input size fits
    batch_size * grid_size

    extern __shared__ int shared[]; // num_stages * block.size() *
    sizeof(int) bytes
    size_t shared_offset[num_stages];
    for (int s = 0; s < num_stages; ++s) shared_offset[s] = s * block.
    size();

    auto block_batch = [&](size_t batch) -> int {
        return block.group_index().x * block.size() + grid.size() * batch;
    };

    // Fill the pipeline with the first `num_stages` batches.
    for (int s = 0; s < num_stages; ++s) {
        __pipeline_memcpy_async(shared + shared_offset[s] + tid, global_in
    + block_batch(s) + tid, cuda::aligned_size_t<4>(sizeof(int)));
        __pipeline_commit();
    }

    // compute_batch: next batch to process
    // fetch_batch: next batch to fetch from global memory
    for (size_t compute_batch = 0, fetch_batch = num_stages; compute_batch
    < batch_size; ++compute_batch, ++fetch_batch) {
        // Wait for the first requested stage to complete.
        constexpr size_t pending_batches = num_stages - 1;
        __pipeline_wait_prior<pending_batches>();
        __syncthreads(); // Not required if each thread works on the data it
    copied.

        // Compute on the current batch.
        compute(global_out + block_batch(compute_batch) + tid, shared +
    shared_offset[stage] + tid);

        __syncthreads(); // Not required if each thread works on the data it
    copied.

        // Load future stage `num_stages` ahead of current compute batch.
        if (fetch_batch < batch_size) {
            __pipeline_memcpy_async(shared + shared_offset[stage] + tid,
    global_in + block_batch(fetch_batch) + tid, cuda::aligned_size_t<4>
    (sizeof(int)));
        }
        __pipeline_commit();
        stage = (stage + 1) % num_stages;
    }
}

```

The `cuda::memcpy_async` implementation demonstrates a multi-stage data prefetching using `cuda::pipeline` (see [Pipelines](#)) with `cuda::memcpy_async`. It:

- ▶ Initializes a pipeline that is local to the thread.
- ▶ Kick-starts the pipeline by scheduling `num_stages` `memcpy_async` operations.
- ▶ Loops over all the batches: it blocks all threads on the completion of the current batch, then performs the computation on the current batch and finally schedules the next `memcpy_async` if there is one.

The `cooperative_groups::memcpy_async` implementation demonstrates multi-stage data prefetching using `cooperative_groups::memcpy_async`. The main difference with the previous implementation is that we do not use a pipeline object, but instead rely on `cooperative_groups::memcpy_async` to schedule the memory transfers in stages under the hood.

The CUDA C primitives implementation demonstrates multi-stage data prefetching using the low-level primitives in a quite similar manner to the first.

An important detail to enable efficient code generation in this example is to keep `num_stages` batches in the pipeline, even if there are no more batches to fetch. This is done by committing to the pipeline even if there are no more batches to fetch (`pipeline.producer_commit()` or `__pipeline_commit()`). Note that this is not possible with the cooperative groups API as we have no access to the internal pipeline.

4.11.1.3 Producer-Consumer Pattern Through Warp Specialization

In this example, we will demonstrate how to implement a producer-consumer pattern where a single warp is specialized as the producer performing asynchronous data copies from global to shared memory, while the remaining warps consume the data from shared memory and perform computations. To enable concurrency between the producer and the consumer threads, we use double-buffering in shared memory. While consumer warps process data in one buffer, the producer warp asynchronously fetches the next batch of data into the other buffer.

CUDA C++ `cuda::memcpy_async`

```

#include <cooperative_groups.h>
#include <cuda/pipeline>

#pragma nv_diag_suppress static_var_with_dynamic_init

using pipeline = cuda::pipeline<cuda::thread_scope_block>;

__device__ void produce(pipeline &pipe, int num_stages, int stage, int num_
→batches, int batch, float *buffer, int buffer_len, float *in, int N)
{
    if (batch < num_batches)
    {
        pipe.producer_acquire();
        /* copy data from in(batch) to buffer(stage) using asynchronous memory
→copies */
        cuda::memcpy_async(buffer + stage * buffer_len + threadIdx.x, in +
→batch * buffer_len + threadIdx.x, cuda::aligned_size_t<4>(sizeof(float)),
→pipe);
        pipe.producer_commit();
    }
}

__device__ void consume(pipeline &pipe, int num_stages, int stage, int num_
→batches, int batch, float *buffer, int buffer_len, float *out, int N)
{
    pipe.consumer_wait();
    /* consume buffer(stage) and update out(batch) */
    pipe.consumer_release();
}

__global__ void producer_consumer_pattern(float *in, float *out, int N, int
→buffer_len)
{
    auto block = cooperative_groups::this_thread_block();
    constexpr int warpSize = 32;

    /* Shared memory buffer declared below is of size 2 * buffer_len
    so that we can alternatively work between two buffers.
    buffer_0 = buffer and buffer_1 = buffer + buffer_len */
    __shared__ extern float buffer[];

    const int num_batches = N / buffer_len;

    /* Create a partitioned pipeline with 2 stages where the first warp is the
→producer and the other warps are consumers.
    constexpr auto scope = cuda::thread_scope_block;
    constexpr int num_stages = 2;
    cuda::std::size_t producer_count = warpSize;
    __shared__ cuda::pipeline_shared_state<scope, num_stages> shared_state;
    pipeline pipe = cuda::make_pipeline(block, &shared_state, producer_count);

    // Producer fills the pipeline
344if (block.thread_rank() < producer_count)
    for (int s = 0; s < num_stages; ++s)
        produce(pipe, num_stages, s, num_batches, s, buffer, buffer_len, in,
→N);

```


CUDA C primitives

```

#include <cooperative_groups.h>
#include <cuda_aware_barrier_primitives.h>

__device__ void produce(__mbarrier_t ready[], __mbarrier_t filled[], float
    ↪ *buffer, int buffer_len, float *in, int N)
{
    for (int i = 0; i < N / buffer_len; ++i)
    {
        __mbarrier_token_t token = __mbarrier_arrive(&ready[i % 2]); /* wait
    ↪ for buffer_(i%2) to be ready to be filled */
        while(!__mbarrier_try_wait(&ready[i % 2], token, 1000)) {}
        /* produce, i.e., fill in, buffer_(i%2) */
        __pipeline_memcpy_async(buffer + i * buffer_len + threadIdx.x, in + i *
    ↪ buffer_len + threadIdx.x, cuda::aligned_size_t<4>(sizeof(float)));
        __pipeline_arrive_on(filled[i % 2]);
        __mbarrier_arrive(filled[i % 2]); /* buffer_(i%2) is filled */
    }
}

__device__ void consume(__mbarrier_t ready[], __mbarrier_t filled[], float
    ↪ *buffer, int buffer_len, float *out, int N)
{
    __mbarrier_arrive(&ready[0]); /* buffer_0 is ready for initial fill */
    __mbarrier_arrive(&ready[1]); /* buffer_1 is ready for initial fill */
    for (int i = 0; i < N / buffer_len; ++i)
    {
        __mbarrier_token_t token = __mbarrier_arrive(&filled[i % 2]);
        while(!__mbarrier_try_wait(&filled[i % 2], token, 1000)) {}
        /* consume buffer_(i%2) */
        __mbarrier_arrive(&ready[i % 2]); /* buffer_(i%2) is ready to be re-
    ↪ filled */
    }
}

__global__ void producer_consumer_pattern(int N, float *in, float *out, int
    ↪ buffer_len)
{
    /* Shared memory buffer declared below is of size 2 * buffer_len
       so that we can alternatively work between two buffers.
       buffer_0 = buffer and buffer_1 = buffer + buffer_len */
    __shared__ extern float buffer[];

    /* bar[0] and bar[1] track if buffers buffer_0 and buffer_1 are ready to be
    ↪ filled,
       while bar[2] and bar[3] track if buffers buffer_0 and buffer_1 are
    ↪ filled-in respectively */
    __shared__ __mbarrier_t bar[4];

    // Initialize the barriers
    auto block = cooperative_groups::this_thread_block();
    if (block.thread_rank() < 4)
346 __mbarrier_init(bar + block.thread_rank(), block.size(), Chapter 4. CUDA Features
    __syncthreads());

    if (block.thread_rank() < warpSize)

```


The `cuda::memcpy_async` implementation demonstrates the API with the highest level of abstraction with `cuda::memcpy_async` and a `cuda::pipeline` with 2 stages. It uses a partitioned pipeline (see [Pipelines](#)) where the first warp serves as a producer and the remaining warps as consumers. Producers initially fill both pipeline stages. Then, in the main processing loop, while consumers process the current batch, producers fetch data for future batches, maintaining a steady flow of work.

The CUDA C primitives implementation based on primitives combines `__pipeline_memcpy_async()` with [shared memory barriers](#) as the completion mechanism to coordinate the asynchronous memory transfers. The `__pipeline_arrive_on()` function associates the memory copy with the barrier. It increments the barrier arrival count by one and when all asynchronous operations sequenced before it have completed, the arrival count is automatically decremented by one and hence the net effect on the arrival count is zero. For this reason, we also need to explicitly arrive at the barrier with `__mbarrier_arrive()`.

4.11.2. Using the Tensor Memory Accelerator (TMA)

Many applications need to move large amounts of data to and from global memory. Often, the data is laid out in global memory as a multi-dimensional array with non-sequential data access patterns. To reduce global memory accesses, sub-tiles of such arrays are copied to shared memory before use in computations. The loading and storing involves address-calculations that can be error-prone and repetitive. To offload these computations, compute capability 9.0 (Hopper) and later (see [PTX documentation](#)) have a *tensor memory accelerator* (TMA). The primary goal of the TMA is to provide an efficient data transfer mechanism from global memory to shared memory for multi-dimensional arrays.

Naming. Tensor memory accelerator (TMA) is a broad term used to refer to the features described in this section. For the purpose of forward-compatibility and to reduce discrepancies with the PTX ISA, the text in this section refers to TMA operations as either *bulk-asynchronous copies* or *bulk-tensor asynchronous copies*, depending on the specific type of copy used. The term “bulk” is used to contrast these operations with the asynchronous memory operations described in the previous section.

Dimensions. TMA supports copying both one-dimensional and multi-dimensional arrays (up to 5-dimensional). The programming model for bulk-asynchronous copies of one-dimensional contiguous arrays is different from the programming model for bulk-tensor asynchronous copies of multi-dimensional arrays. To perform a bulk-tensor asynchronous copy of a multi-dimensional array, the hardware requires a [tensor map](#). This object describes the layout of the multi-dimensional array in global and shared memory. A tensor map is typically created on the host using the [cuTensorMapEncode API](#) and then transferred from host to device as a const kernel parameter annotated with `__grid_constant__` (see [__grid_constant__ Parameters](#)). The tensor map is transferred from host to device as a const kernel parameter annotated with `__grid_constant__`, and can be used on the device to copy a tile of data between shared and global memory. In contrast, performing a bulk-asynchronous copy of a contiguous one-dimensional array does not require a tensor map: it can be performed on-device with a pointer and size parameter.

Source and destination. The source and destination addresses of TMA operations can be in shared or global memory. The operations can read data from global to shared memory, write data from shared to global memory, and also copy from shared memory to [distributed shared memory](#) of another block in the same cluster. In addition, when in a cluster, a bulk-asynchronous tensor operation can be specified as being *multicast*. In this case, data can be transferred from global memory to the shared memory of multiple blocks within the cluster. The multicast feature is optimized for target architecture `sm_90a` and may have [significantly reduced performance](#) on other targets. Hence, it is advised to be used with [compute architecture](#) `sm_90a`.

Asynchronicity. Data transfers using TMA are [asynchronous](#) and are modeled as async proxy operations (see [Async Thread and Async Proxy](#)). This allows the initiating thread to continue computing

while the hardware asynchronously copies the data. *Whether the data transfer occurs asynchronously in practice is up to the hardware implementation and may change in the future.* There are several **completion mechanisms** that bulk-asynchronous operations can use to signal that they have completed. When the operation reads from global to shared memory, any thread in the block can wait for the data to be readable in shared memory by waiting on a **shared memory barrier**. When the bulk-asynchronous operation writes data from shared memory to global or distributed shared memory, only the initiating thread can wait for the operation to have completed. This is accomplished using a **bulk async-group** based completion mechanism. A table describing the completion mechanisms can be found below and in the **PTX ISA**.

Table 19: Asynchronous copies with possible source and destination memory spaces and completion mechanisms using TMA. An empty cell indicates that a source-destination pair is not supported.

Direction		Asynchronous Copy (TMA, CC 9.0+)
Source	Destination	Completion Mechanism
global	global	
shared::cta	global	bulk async-group
global	shared::cta	shared memory barrier
global	shared::cluster	shared memory barrier (multicast)
shared::cta	shared::cluster	shared memory barrier
shared::cta	shared::cta	

4.11.2.1 Using TMA to transfer one-dimensional arrays

The following table summarizes the possible source and destination memory spaces and completion mechanisms for bulk-asynchronous TMA along with the API that exposes it.

Table 20: Asynchronous copies with possible source and destination memory spaces and completion mechanisms using bulk-asynchronous TMA. An empty cell indicates that a source-destination pair is not supported.

Direction		Bulk-Asynchronous Copy (TMA, CC9.0+)	
Source	Destination	Completion Mechanism	API
global	global		
shared::c	global	bulk async-group	<code>cuda::ptx::cp_async_bulk</code>
global	shared::cta	shared memory barrier	<code>cuda::memcpy_async</code> , <code>cuda::device::memcpy_async_tx</code> , <code>cuda::ptx::cp_async_bulk</code>
global	shared::clus	shared memory barrier	<code>cuda::ptx::cp_async_bulk</code>
shared::c	shared::clus	shared memory barrier	<code>cuda::ptx::cp_async_bulk</code>
shared::c	shared::cta		

Some functionality requires inline PTX that is currently made available through the `cuda::ptx` namespace in the [CUDA Standard C++](#) library. The availability of these wrappers can be checked with the following code:

```
#if defined(__CUDA_MINIMUM_ARCH__) && __CUDA_MINIMUM_ARCH__ < 900
static_assert(false, "Device code is being compiled with older architectures
→ that are incompatible with TMA.");
#endif // __CUDA_MINIMUM_ARCH__
```

Note that `cuda::memcpy_async` uses TMA if the source and destination addresses are 16-byte aligned and the size is a multiple of 16 bytes, otherwise it falls back to synchronous copies. On the other hand, `cuda::device::memcpy_async_tx` and `cuda::ptx::cp_async_bulk` always use TMA and will result in undefined behavior if the requirements are not met.

In the following, we demonstrate how to use bulk-asynchronous copies through an example. The example read-modify-writes a one-dimensional array. The kernel goes through the following steps:

1. Initialize a shared memory barrier as a completion mechanism for the bulk-asynchronous copy from global to shared memory.
2. Initiate the copy of a block of memory from global to shared memory.
3. Arrive and wait on the shared memory barrier for completion of the copy.
4. Increment the shared memory buffer values.
5. Use a proxy fence to ensure shared memory writes (generic proxy) become visible to the subsequent bulk-asynchronous copy (async proxy).
6. Initiate a bulk-asynchronous copy of the buffer in shared memory to global memory.
7. Wait for the bulk-asynchronous copy to have finished reading shared memory.

```

#include <cuda/barrier>
#include <cuda/ptx>

using barrier = cuda::barrier<cuda::thread_scope_block>;
namespace ptx = cuda::ptx;

static constexpr size_t buf_len = 1024;

__device__ inline bool is_elected()
{
    unsigned int tid = threadIdx.x;
    unsigned int warp_id = tid / 32;
    unsigned int uniform_warp_id = __shfl_sync(0xFFFFFFFF, warp_id, 0); //
    ↪ Broadcast from lane 0.
    return (uniform_warp_id == 0 && ptx::elect_sync(0xFFFFFFFF)); // Elect a
    ↪ leader thread among warp 0.
}

__global__ void add_one_kernel(int* data, size_t offset)
{
    // Shared memory buffer. The destination shared memory buffer of
    // a bulk operation should be 16 byte aligned.
    __shared__ alignas(16) int smem_data[buf_len];

    // 1. Initialize shared memory barrier with the number of threads
    ↪ participating in the barrier.
    #pragma nv_diag_suppress static_var_with_dynamic_init
    __shared__ barrier bar;
    if (threadIdx.x == 0) {
        init(&bar, blockDim.x);
    }
    __syncthreads();

    // 2. Initiate TMA transfer to copy global to shared memory from a single
    ↪ thread.
    if (is_elected()) {
        // Launch the async copy and communicate how many bytes are expected to
        ↪ come in (the transaction count).

        // Version 1: cuda::memcpy_async
        cuda::memcpy_async(
            smem_data, data + offset,
            cuda::aligned_size_t<16>(sizeof(smem_data)),
            bar);

        // Version 2: cuda::device::memcpy_async_tx
        // cuda::device::memcpy_async_tx(
        //     smem_data, data + offset,
        //     cuda::aligned_size_t<16>(sizeof(smem_data)),
        //     bar);
        // cuda::device::barrier_expect_tx(
        //     cuda::device::barrier_native_handle(bar),
        //     sizeof(smem_data));
    }
}

```

(continues on next page)

(continued from previous page)

```

// Version 3: cuda::ptx::cp_async_bulk
// ptx::cp_async_bulk(
//     ptx::space_shared, ptx::space_global,
//     smem_data, data + offset,
//     sizeof(smem_data),
//     cuda::device::barrier_native_handle(bar));
// cuda::device::barrier_expect_tx(
//     cuda::device::barrier_native_handle(bar),
//     sizeof(smem_data));
}

// 3a. All threads arrive on the barrier.
barrier::arrival_token token = bar.arrive();

// 3b. Wait for the data to have arrived.
bar.wait(std::move(token));

// 4. Compute saxpy and write back to shared memory.
for (int i = threadIdx.x; i < buf_len; i += blockDim.x) {
    smem_data[i] += 1;
}

// 5. Wait for shared memory writes to be visible to TMA engine.
ptx::fence_proxy_async(ptx::space_shared);
__syncthreads();
// After syncthreads, writes by all threads are visible to TMA engine.

// 6. Initiate TMA transfer to copy shared memory to global memory.
if (is_elected()) {
    ptx::cp_async_bulk(
        ptx::space_global, ptx::space_shared,
        data + offset, smem_data, sizeof(smem_data));
    // 7. Wait for TMA transfer to have finished reading shared memory.
    // Create a "bulk async-group" out of the previous bulk copy operation.
    ptx::cp_async_bulk_commit_group();
    // Wait for the group to have completed reading from shared memory.
    ptx::cp_async_bulk_wait_group_read(ptx::n32_t<0>());
}
}

```

Barrier initialization. The barrier is initialized with the number of threads participating in the block. As a result, the barrier will flip only if all threads have arrived on this barrier. Shared memory barriers are described in more detail in [shared memory barriers](#).

TMA read. The bulk-asynchronous copy instruction directs the hardware to copy a large chunk of data into shared memory, and to update the [transaction count](#) of the shared memory barrier after completing the read. In general, issuing as few bulk copies with as big a size as possible results in the best performance. Because the copy can be performed asynchronously by the hardware, it is not necessary to split the copy into smaller chunks.

The thread that initiates the bulk-asynchronous copy operation also tells the barrier how many transactions (tx) are expected to arrive. In this case, the transactions are counted in bytes. This is automatically performed by `cuda::memcpy_async`, but not by

`cuda::device::memcpy_async_tx` and `cuda::ptx::cp_async_bulk` after which we need to explicitly call `cuda::ptx::mbarrier_expect_tx`. If multiple threads update the transaction count, the expected transaction count will be the sum of the updates. The barrier will only flip once all threads have arrived **and** all bytes have arrived. Once the barrier has flipped, the bytes are safe to read from shared memory, both by the threads as well as by subsequent bulk-asynchronous copies. More information about barrier transaction accounting can be found in [Tracking Asynchronous Memory Operations](#).

Barrier wait. Waiting for the barrier to flip is done using tokens with `bar.wait()`. It can be more efficient to use explicit phase tracking of the barrier (see [Explicit Phase Tracking](#)).

SMEM write and sync. The increment of the buffer values reads and writes to shared memory. To make the writes visible to subsequent bulk-asynchronous copies, the `cuda::ptx::fence_proxy_async` function is used. This orders the writes to shared memory before subsequent reads from bulk-asynchronous copy operations, which read through the async proxy. So each thread first orders the writes to objects in shared memory in the async proxy via the `cuda::ptx::fence_proxy_async`, and these operations by all threads are ordered before the async operation performed in thread 0 using `__syncthreads()`.

TMA write and sync. The write from shared to global memory is again initiated by a single thread. The completion of the write is not tracked by a shared memory barrier. Instead, a thread-local mechanism is used. Multiple writes can be batched into a so-called *bulk async-group*. Afterwards, the thread can wait for all operations in this group to have completed reading from shared memory (as in the code above) or to have completed writing to global memory, making the writes visible to the initiating thread. For more information, refer to the PTX ISA documentation of [cp.async.bulk.wait_group](#). Note that the bulk-asynchronous and non-bulk-asynchronous copy instructions have different async-groups: there exist both `cp.async.wait_group` and `cp.async.bulk.wait_group` instructions.

Note

It is recommended to initiate TMA operations by a single thread in the block. While using `if (threadIdx.x == 0)` might seem sufficient, the compiler cannot verify that indeed only one thread is initiating the copy and may insert a peeling loop over all active threads, which results in warp serialization and reduced performance. To prevent this, we define the `is_elected()` helper function that uses `cuda::ptx::elect_sync` to select one thread from warp 0 – which is known to the compiler – to execute the copy allowing it to generate more efficient code. Alternatively, the same effect can be achieved with [cooperative_groups::invoke_one](#).

The bulk-asynchronous instructions have specific alignment requirements on their source and destination addresses. More information can be found in the table below.

Table 21: Alignment requirements for one-dimensional bulk-asynchronous operations.

Address / Size	Alignment
Global memory address	Must be 16 byte aligned.
Shared memory address	Must be 16 byte aligned.
Shared memory barrier address	Must be 8 byte aligned (this is guaranteed by <code>cuda::barrier</code>).
Size of transfer	Must be a multiple of 16 bytes.

4.11.2.1.1 Prefetching Data

In this example, we will demonstrate how to use TMA to prefetch data from global memory to shared memory. In an iterative copy and compute pattern, this allows hiding the latency of data transfers of future iterations with computation on the current iteration, potentially increasing bytes-in-flight.

CUDA C++ `cuda::device::memcpy_async_tx`

```

#include <cooperative_groups.h>
#include <cuda/barrier>
#include <cuda/ptx>

namespace ptx = cuda::ptx;
namespace cg = cooperative_groups;

__device__ inline bool is_elected()
{
    unsigned int tid = threadIdx.x;
    unsigned int warp_id = tid / 32;
    unsigned int uniform_warp_id = __shfl_sync(0xFFFFFFFF, warp_id, 0); //
    ↪ Broadcast from lane 0.
    return (uniform_warp_id == 0 && ptx::elect_sync(0xFFFFFFFF)); // Elect
    ↪ a leader thread among warp 0.
}

template <int block_size, int num_stages>
__global__ void prefetch_kernel(int* global_out, int const* global_in, size_
    ↪ t size, size_t batch_size) {
    auto grid = cg::this_grid();
    auto block = cg::this_thread_block();
    const int tid = threadIdx.x;
    assert(size == batch_size * grid.size()); // Assume input size fits
    ↪ batch_size * grid_size

    // 1. Initialization Phase
    __shared__ int shared[num_stages * block_size];
    size_t shared_offset[num_stages];
    for (int s = 0; s < num_stages; ++s) shared_offset[s] = s * block.
    ↪ size();

    auto block_batch = [&](size_t batch) -> int {
        return block.group_index().x * block.size() + grid.size() * batch;
    };

    // Initialize shared memory barrier with the number of threads
    ↪ participating in the barrier.
    // We will use explicit phase tracking for the barrier, which allows us
    ↪ to have only one
    // thread arrive on the barrier to set the transaction count and other
    ↪ threads wait for
    // a parity-based phase flip.
    #pragma nv_diag_suppress static_var_with_dynamic_init
    __shared__ cuda::barrier<cuda::thread_scope_block> bar[num_stages];
    if (tid == 0) {
        #pragma unroll num_stages
        for (int i = 0; i < num_stages; i++) {
            init(&bar[i], 1);
        }
    }
    __syncthreads();

    // Fill the pipeline with the first ``num_stages`` batches.
    if (is_elected()) {
        size_t num_bytes = block_size * sizeof(int);

```


This example implements *multi-stage data prefetching* using `cuda::device::memcpy_async_tx` for the TMA copies and employs shared memory barriers with explicit phase tracking for synchronization of the copies.

1. **Initialization Phase:** Sets up shared memory barriers (one per stage) and pre-loads the first `num_stages` batches into different shared memory sections.
2. **Main Processing Loop:**
 - a. **Wait:** Uses `mbarrier_try_wait_parity()` to wait for the current batch to complete copying.
 - b. **Compute:** Processes the current batch data.
 - c. **Prefetch:** Schedules the next `memcpy_async_tx` operation for future data (staying `num_stages` ahead).
 - d. **Stage Management:** Cycles through stages using a rotating buffer approach and tracks barrier parity.

4.11.2.2 Using TMA to transfer multi-dimensional arrays

In this section, we will focus on multi-dimensional TMA copies. The primary difference between the one-dimensional and multi-dimensional case is that a tensor map must be created on the host and passed to the CUDA kernel.

The following table summarizes the possible source and destination memory spaces and completion mechanisms for bulk-tensor asynchronous TMA along with the API that exposes it in device code.

Table 22: Asynchronous copies with possible source and destination memory spaces and completion mechanisms using bulk-tensor asynchronous TMA. An empty cell indicates that a source-destination pair is not supported.

Direction		Bulk-Tensor Asynchronous Copy (TMA, CC9.0+)	
Source	Destination	Completion Mechanism	API
global	global		
shared::cta	global	bulk async-group	<code>cuda::ptx::cp_async_bulk_tensor</code>
global	shared::cta	shared memory barrier	<code>cuda::ptx::cp_async_bulk_tensor</code>
global	shared::cluster	shared memory barrier	<code>cuda::ptx::cp_async_bulk_tensor</code>
shared::cta	shared::cluster	shared memory barrier	<code>cuda::ptx::cp_async_bulk_tensor</code>
shared::cta	shared::cta		

All functionality requires inline PTX that is currently made available through the `cuda::ptx` namespace in the [CUDA Standard C++](#) library.

In the following, we describe how to create a tensor map using the CUDA driver API, how to pass it to the device, and how to use it on the device.

Driver API. A tensor map is created using the `cuTensorMapEncodeTiled` driver API. This API can be accessed by linking to the driver directly (`-lcuda`) or by using the `cudaGetDriverEntryPointByVersion` API. Below, we show how to get a pointer to the `cuTensorMapEncodeTiled` API. For more information, refer to [Driver Entry Point Access](#).

```
#include <cudaTypedefs.h> // PFN_cuTensorMapEncodeTiled, CUtensorMap

PFN_cuTensorMapEncodeTiled_v12000 get_cuTensorMapEncodeTiled() {
    // Get pointer to cuTensorMapEncodeTiled
    cudaDriverEntryPointQueryResult driver_status;
    void* cuTensorMapEncodeTiled_ptr = nullptr;
    CUDA_CHECK(cudaGetDriverEntryPointByVersion("cuTensorMapEncodeTiled", &
    ↪ cuTensorMapEncodeTiled_ptr, 12000, cudaEnableDefault, &driver_status));
    assert(driver_status == cudaDriverEntryPointSuccess);

    return reinterpret_cast<PFN_cuTensorMapEncodeTiled_v12000>
    ↪(cuTensorMapEncodeTiled_ptr);
}
```

Creation. Creating a tensor map requires many parameters. Among them are the base pointer to an array in global memory, the size of the array (in number of elements), the stride from one row to the next (in bytes), the size of the shared memory buffer (in number of elements). The code below creates a tensor map to describe a two-dimensional row-major array of size GMEM_HEIGHT x GMEM_WIDTH. Note the order of the parameters: the fastest moving dimension comes first.

```
CUtensorMap tensor_map{};
// rank is the number of dimensions of the array.
constexpr uint32_t rank = 2;
uint64_t size[rank] = {GMEM_WIDTH, GMEM_HEIGHT};
// The stride is the number of bytes to traverse from the first element of
↪ one row to the next.
// It must be a multiple of 16.
uint64_t stride[rank - 1] = {GMEM_WIDTH * sizeof(int)};
// The box_size is the size of the shared memory buffer that is used as the
// destination of a TMA transfer.
uint32_t box_size[rank] = {SMEM_WIDTH, SMEM_HEIGHT};
// The distance between elements in units of sizeof(element). A stride of 2
// can be used to load only the real component of a complex-valued tensor,
↪ for instance.
uint32_t elem_stride[rank] = {1, 1};

// Get a function pointer to the cuTensorMapEncodeTiled driver API.
auto cuTensorMapEncodeTiled = get_cuTensorMapEncodeTiled();

// Create the tensor descriptor.
CUresult res = cuTensorMapEncodeTiled(
    &tensor_map,           // CUtensorMap *tensorMap,
    CUtensorMapDataType::CU_TENSOR_MAP_DATA_TYPE_INT32,
    rank,                 // cuuint32_t tensorRank,
    tensor_ptr,           // void *globalAddress,
    size,                 // const cuuint64_t *globalDim,
    stride,               // const cuuint64_t *globalStrides,
    box_size,             // const cuuint32_t *boxDim,
    elem_stride,          // const cuuint32_t *elementStrides,
    // Interleave patterns can be used to accelerate loading of values that
    // are less than 4 bytes long.
    CUtensorMapInterleave::CU_TENSOR_MAP_INTERLEAVE_NONE,
    // Swizzling can be used to avoid shared memory bank conflicts.
```

(continues on next page)

(continued from previous page)

```

    CUTensorMapSwizzle::CU_TENSOR_MAP_SWIZZLE_NONE,
    // L2 Promotion can be used to widen the effect of a cache-policy to a
    →wider
    // set of L2 cache lines.
    CUTensorMapL2promotion::CU_TENSOR_MAP_L2_PROMOTION_NONE,
    // Any element that is outside of bounds will be set to zero by the TMA
    →transfer.
    CUTensorMapFloat00Bfill::CU_TENSOR_MAP_FLOAT_00B_FILL_NONE
);

```

Host-to-device transfer. There are three ways to make a tensor map accessible to device code. The recommended approach is to pass the tensor map as a `const __grid_constant__` parameter to a kernel. The other possibilities are copying the tensor map into device `__constant__` memory using `cudaMemcpyToSymbol` or accessing it via global memory. When passing the tensor map as a parameter, some versions of the GCC C++ compiler issue the warning “the ABI for passing parameters with 64-byte alignment has changed in GCC 4.6”. This warning can be ignored.

```

#include <cuda.h>

__global__ void kernel(const __grid_constant__ CUTensorMap tensor_map)
{
    // Use tensor_map here.
}

int main() {
    CUTensorMap map;
    // [ ..Initialize map.. ]
    kernel<<<1, 1>>>(map);
}

```

As an alternative to the `__grid_constant__` kernel parameter, a global `__constant__` variable can be used. An example is included below.

```

#include <cuda.h>

__constant__ CUTensorMap global_tensor_map;
__global__ void kernel()
{
    // Use global_tensor_map here.
}

int main() {
    CUTensorMap local_tensor_map;
    // [ ..Initialize map.. ]
    cudaMemcpyToSymbol(global_tensor_map, &local_tensor_map,
    →sizeof(CUTensorMap));
    kernel<<<1, 1>>>();
}

```

Finally, it is possible to copy the tensor map to global memory. Using a pointer to a tensor map in global device memory requires a fence in each thread block before any thread in the block uses the updated tensor map. Further uses of the tensor map by that thread block do not need to be fenced unless the tensor map is modified again. Note that this mechanism may be slower than the two mechanisms described above.

```

#include <cuda.h>
#include <cuda/ptx>
namespace ptx = cuda::ptx;

__device__ CUTensorMap global_tensor_map;
__global__ void kernel(CUTensorMap *tensor_map)
{
    // Fence acquire tensor map:
    ptx::n32_t<128> size_bytes;
    // Since the tensor map was modified from the host using cudaMemcpy,
    // the scope should be .sys.
    ptx::fence_proxy_tensormap_generic(
        ptx::sem_acquire, ptx::scope_sys, tensor_map, size_bytes
    );
    // Safe to use tensor_map after fence inside this thread.
}

int main() {
    CUTensorMap local_tensor_map;
    // [ ..Initialize map.. ]
    cudaMemcpy(&global_tensor_map, &local_tensor_map, sizeof(CUTensorMap),
        ↪ cudaMemcpyHostToDevice);
    kernel<<<1, 1>>>>(global_tensor_map);
}

```

Use. The kernel below loads a 2D tile of size `SMEM_HEIGHT` x `SMEM_WIDTH` from a larger 2D array. The top-left corner of the tile is indicated by the indices `x` and `y`. The tile is loaded into shared memory, modified, and written back to global memory.

```

#include <cuda.h>           // CUTensormap
#include <cuda/barrier>

using barrier = cuda::barrier<cuda::thread_scope_block>;
namespace ptx = cuda::ptx;

__device__ inline bool is_elected()
{
    unsigned int tid = threadIdx.x;
    unsigned int warp_id = tid / 32;
    unsigned int uniform_warp_id = __shfl_sync(0xFFFFFFFF, warp_id, 0); //
    ↪ Broadcast from lane 0.
    return (uniform_warp_id == 0 && ptx::elect_sync(0xFFFFFFFF)); // Elect a
    ↪ leader thread among warp 0.
}

__global__ void kernel(const __grid_constant__ CUTensorMap tensor_map, int x,
    ↪ int y) {
    // The destination shared memory buffer of a bulk tensor operation should be
    // 128 byte aligned.
    __shared__ alignas(128) int smem_buffer[SMEM_HEIGHT][SMEM_WIDTH];

    // Initialize shared memory barrier with the number of threads participating
    ↪ in the barrier.
    #pragma nv_diag_suppress static_var_with_dynamic_init

```

(continues on next page)

(continued from previous page)

```

__shared__ barrier bar;

if (threadIdx.x == 0) {
    // Initialize barrier. All `blockDim.x` threads in block participate.
    init(&bar, blockDim.x);
}
// Syncthreads so initialized barrier is visible to all threads.
__syncthreads();

barrier::arrival_token token;
if (is_elected()) {
    // Initiate bulk tensor copy.
    int32_t tensor_coords[2] = { x, y };
    ptx::cp_async_bulk_tensor(
        ptx::space_shared, ptx::space_global,
        &smem_buffer, &tensor_map, tensor_coords,
        cuda::device::barrier_native_handle(bar));
    // Arrive on the barrier and tell how many bytes are expected to come in.
    token = cuda::device::barrier_arrive_tx(bar, 1, sizeof(smem_buffer));
} else {
    // Other threads just arrive.
    token = bar.arrive();
}
// Wait for the data to have arrived.
bar.wait(std::move(token));

// Symbolically modify a value in shared memory.
smem_buffer[0][threadIdx.x] += threadIdx.x;

// Wait for shared memory writes to be visible to TMA engine.
ptx::fence_proxy_async(ptx::space_shared);
__syncthreads();
// After syncthreads, writes by all threads are visible to TMA engine.

// Initiate TMA transfer to copy shared memory to global memory
if (is_elected()) {
    int32_t tensor_coords[2] = { x, y };
    ptx::cp_async_bulk_tensor(
        ptx::space_global, ptx::space_shared,
        &tensor_map, tensor_coords, &smem_buffer);
    // Wait for TMA transfer to have finished reading shared memory.
    // Create a "bulk async-group" out of the previous bulk copy operation.
    ptx::cp_async_bulk_commit_group();
    // Wait for the group to have completed reading from shared memory.
    ptx::cp_async_bulk_wait_group_read(ptx::n32_t<0>());
}

// Destroy barrier. This invalidates the memory region of the barrier. If
// further computations were to take place in the kernel, this allows the
// memory location of the shared memory barrier to be reused.
if (threadIdx.x == 0) {
    (&bar)->~barrier();
}

```

(continues on next page)

(continued from previous page)

```

    }
}

```

Negative indices and out of bounds. When part of the tile that is being *read* from global to shared memory is out of bounds, the shared memory that corresponds to the out of bounds area is zero-filled. The top-left corner indices of the tile may also be negative. When *writing* from shared to global memory, parts of the tile may be out of bounds, but the top left corner cannot have any negative indices.

Size and stride. The size of a tensor is the number of elements along one dimension. All sizes must be greater than one. The stride is the number of bytes between elements of the same dimension. For instance, a 4×4 matrix of integers has sizes 4 and 4. Since it has 4 bytes per element, the strides are 4 and 16 bytes. Due to alignment requirements, a 4×3 row-major matrix of integers must have strides of 4 and 16 bytes as well. Each row is padded with 4 extra bytes to ensure that the start of the next row is aligned to 16 bytes. More information about alignment requirements can be found in the table below.

Table 23: Alignment requirements for multi-dimensional bulk tensor asynchronous copy operations.

Address / Size	Alignment
Global memory address	Must be 16 byte aligned.
Global memory sizes	Must be greater than or equal to one. Does not have to be a multiple of 16 bytes.
Global memory strides	Must be multiples of 16 bytes.
Shared memory address	Must be 128 byte aligned.
Shared memory barrier address	Must be 8 byte aligned (this is guaranteed by <code>cuda::barrier</code>).
Size of transfer	Must be a multiple of 16 bytes.

4.11.2.2.1 Encoding a Tensor Map on Device

Previous sections have described how to create a tensor map on the host using the CUDA driver API.

This section explains how to encode a tiled-type tensor map on device. This is useful in situations where the typical way of transferring the tensor map (using `const __grid_constant__` kernel parameters) is undesirable, for instance, when processing a batch of tensors of various sizes in a single kernel launch.

The recommended pattern is as follows:

1. Create a tensor map “template”, `template_tensor_map`, using the Driver API on the host.
2. In a device kernel, copy the `template_tensor_map`, modify the copy, store in global memory, and appropriately fence.
3. Use the tensor map in a kernel with appropriate fencing.

The high-level code structure is as follows:

```

// Initialize device context:
CUDA_CHECK(cudaDeviceSynchronize());

// Create a tensor map template using the cuTensorMapEncodeTiled driver function
CUtensorMap template_tensor_map = make_tensormap_template();

// Allocate tensor map and tensor in global memory
CUtensorMap* global_tensor_map;
CUDA_CHECK(cudaMalloc(&global_tensor_map, sizeof(CUtensorMap)));
char* global_buf;
CUDA_CHECK(cudaMalloc(&global_buf, 8 * 256));

// Fill global buffer with data.
fill_global_buf<<<1, 1>>>(global_buf);

// Define the parameters of the tensor map that will be created on device.
tensormap_params p{};
p.global_address = global_buf;
p.rank = 2;
p.box_dim[0] = 128; // The box in shared memory has half the width of
    ↳ the full buffer
p.box_dim[1] = 4; // The box in shared memory has half the height of
    ↳ the full buffer
p.global_dim[0] = 256; //
p.global_dim[1] = 8; //
p.global_stride[0] = 256; //
p.element_stride[0] = 1; //
p.element_stride[1] = 1; //

// Encode global_tensor_map on device:
encode_tensor_map<<<1, 32>>>(template_tensor_map, p, global_tensor_map);

// Use it from another kernel:
consume_tensor_map<<<1, 1>>>(global_tensor_map);

// Check for errors:
CUDA_CHECK(cudaDeviceSynchronize());

```

The following sections describe the high-level steps. Throughout the examples, the following `tensormap_params` struct contains the new values of the fields to be updated. It is included here to reference when reading the examples.

```

struct tensormap_params {
    void* global_address;
    int rank;
    uint32_t box_dim[5];
    uint64_t global_dim[5];
    size_t global_stride[4];
    uint32_t element_stride[5];
};

```

4.11.2.2.2 Device-side Encoding and Modification of a Tensor Map

The recommended process of encoding a tensor map in global memory proceeds as follows.

1. Pass an existing tensor map, the `template_tensor_map`, to the kernel. In contrast to kernels that use the tensor map in a `cp.async.bulk.tensor` instruction, this may be done in any way: a pointer to global memory, kernel parameter, a `__constant__` variable, and so on.
2. Copy-initialize a tensor map in shared memory with the `template_tensor_map` value.
3. Modify the tensor map in shared memory using the `cuda::ptx::tensormap_replace` functions. These functions wrap the `tensormap.replace` PTX instruction, which can be used to modify any field of a tiled-type tensor map, including the base address, size, stride, and so on.
4. Using the `cuda::ptx::tensormap_copy_fenceproxy` function, copy the modified tensor map from shared memory to global memory and perform any necessary fencing.

The following code contains a kernel that follows these steps. For completeness, it modifies all the fields of the tensor map. Typically, a kernel will modify just a few fields.

In this kernel, `template_tensor_map` is passed as a kernel parameter. This is the preferred way of moving `template_tensor_map` from the host to the device. If the kernel is intended to update an existing tensor map in device memory, it can take a pointer to the existing tensor map to modify.

Note

The format of the tensor map may change over time. Therefore, the `cuda::ptx::tensormap_replace` functions and corresponding `tensormap.replace.tile` PTX instructions are marked as specific to sm_90a. To use them, compile using `nvcc -arch sm_90a . . . .`

Tip

On sm_90a, a zero-initialized buffer in shared memory may also be used as the initial tensor map value. This enables encoding a tensor map purely on device, without using the driver API to encode the `template_tensor_map` value.

Note

On-device modification is only supported for tiled-type tensor maps; other tensor map types cannot be modified on device. For more information on the tensor map types, refer to the [Driver API reference](#).

```
#include <cuda/ptx>

namespace ptx = cuda::ptx;

// launch with 1 warp.
__launch_bounds__(32)
__global__ void encode_tensor_map(const __grid_constant__ CUtensorMap
    template_tensor_map, tensormap_params p, CUtensorMap* out) {
    __shared__ alignas(128) CUtensorMap smem_tmap;
    if (threadIdx.x == 0) {
```

(continues on next page)

(continued from previous page)

```

// Copy template to shared memory:
smem_tmap = template_tensor_map;

const auto space_shared = ptx::space_shared;
ptx::tensormap_replace_global_address(space_shared, &smem_tmap, p.
→global_address);
// For field .rank, the operand new_val must be ones less than the
→desired
// tensor rank as this field uses zero-based numbering.
ptx::tensormap_replace_rank(space_shared, &smem_tmap, p.rank - 1);

// Set box dimensions:
if (0 < p.rank) { ptx::tensormap_replace_box_dim(space_shared, &smem_
→tmap, ptx::n32_t<0>{}, p.box_dim[0]); }
if (1 < p.rank) { ptx::tensormap_replace_box_dim(space_shared, &smem_
→tmap, ptx::n32_t<1>{}, p.box_dim[1]); }
if (2 < p.rank) { ptx::tensormap_replace_box_dim(space_shared, &smem_
→tmap, ptx::n32_t<2>{}, p.box_dim[2]); }
if (3 < p.rank) { ptx::tensormap_replace_box_dim(space_shared, &smem_
→tmap, ptx::n32_t<3>{}, p.box_dim[3]); }
if (4 < p.rank) { ptx::tensormap_replace_box_dim(space_shared, &smem_
→tmap, ptx::n32_t<4>{}, p.box_dim[4]); }
// Set global dimensions:
if (0 < p.rank) { ptx::tensormap_replace_global_dim(space_shared, &smem_
→tmap, ptx::n32_t<0>{}, (uint32_t) p.global_dim[0]); }
if (1 < p.rank) { ptx::tensormap_replace_global_dim(space_shared, &smem_
→tmap, ptx::n32_t<1>{}, (uint32_t) p.global_dim[1]); }
if (2 < p.rank) { ptx::tensormap_replace_global_dim(space_shared, &smem_
→tmap, ptx::n32_t<2>{}, (uint32_t) p.global_dim[2]); }
if (3 < p.rank) { ptx::tensormap_replace_global_dim(space_shared, &smem_
→tmap, ptx::n32_t<3>{}, (uint32_t) p.global_dim[3]); }
if (4 < p.rank) { ptx::tensormap_replace_global_dim(space_shared, &smem_
→tmap, ptx::n32_t<4>{}, (uint32_t) p.global_dim[4]); }
// Set global stride:
if (1 < p.rank) { ptx::tensormap_replace_global_stride(space_shared, &
→smem_tmap, ptx::n32_t<0>{}, p.global_stride[0]); }
if (2 < p.rank) { ptx::tensormap_replace_global_stride(space_shared, &
→smem_tmap, ptx::n32_t<1>{}, p.global_stride[1]); }
if (3 < p.rank) { ptx::tensormap_replace_global_stride(space_shared, &
→smem_tmap, ptx::n32_t<2>{}, p.global_stride[2]); }
if (4 < p.rank) { ptx::tensormap_replace_global_stride(space_shared, &
→smem_tmap, ptx::n32_t<3>{}, p.global_stride[3]); }
// Set element stride:
if (0 < p.rank) { ptx::tensormap_replace_element_size(space_shared, &
→smem_tmap, ptx::n32_t<0>{}, p.element_stride[0]); }
if (1 < p.rank) { ptx::tensormap_replace_element_size(space_shared, &
→smem_tmap, ptx::n32_t<1>{}, p.element_stride[1]); }
if (2 < p.rank) { ptx::tensormap_replace_element_size(space_shared, &
→smem_tmap, ptx::n32_t<2>{}, p.element_stride[2]); }
if (3 < p.rank) { ptx::tensormap_replace_element_size(space_shared, &
→smem_tmap, ptx::n32_t<3>{}, p.element_stride[3]); }
if (4 < p.rank) { ptx::tensormap_replace_element_size(space_shared, &

```

(continues on next page)

(continued from previous page)

```

→smem_tmap, ptx::n32_t<4>{}, p.element_stride[4]); }

    // These constants are documented in this table:
    // https://docs.nvidia.com/cuda/parallel-thread-execution/index.html
→#tensormap-new-val-validity
    auto u8_elem_type = ptx::n32_t<0>{};
    ptx::tensormap_replace_elemtype(space_shared, &smem_tmap, u8_elem_type);
    auto no_interleave = ptx::n32_t<0>{};
    ptx::tensormap_replace_interleave_layout(space_shared, &smem_tmap, no_
→interleave);
    auto no_swizzle = ptx::n32_t<0>{};
    ptx::tensormap_replace_swizzle_mode(space_shared, &smem_tmap, no_
→swizzle);
    auto zero_fill = ptx::n32_t<0>{};
    ptx::tensormap_replace_fill_mode(space_shared, &smem_tmap, zero_fill);
}
    // Synchronize the modifications with other threads in warp
    __syncwarp();
    // Copy the tensor map to global memory collectively with threads in the
→warp.
    // In addition: make the updated tensor map visible to other threads on
→device that
    // for use with cp.async.bulk.
    ptx::n32_t<128> bytes_128;
    ptx::tensormap_cp_fenceproxy(ptx::sem_release, ptx::scope_gpu, out, &smem_
→tmap, bytes_128);
}

```

4.11.2.2.3 Usage of a Modified Tensor Map

In contrast to using a tensor map that is passed as a const `__grid_constant__` kernel parameter, using a tensor map in global memory requires explicitly establishing a release-acquire pattern in the tensor map proxy between the threads that modify the tensor map and the threads that use it.

The release part of the pattern was shown in the previous section. It is accomplished using the `cuda::ptx::tensormap.cp_fenceproxy` function.

The acquire part is accomplished using the `cuda::ptx::fence_proxy_tensormap_generic` function that wraps the `fence.proxy.tensormap::generic.acquire` instruction. If the two threads participating in the release-acquire pattern are on the same device, the `.gpu` scope suffices. If the threads are on different devices, the `.sys` scope must be used. Once a tensor map has been acquired by one thread, it can be used by other threads in the block after sufficient synchronization, for example, using `__syncthreads()`. The thread that uses the tensor map and the thread that performs the fence must be in the same block. That is, if the threads are in, for example, two different thread blocks of the same cluster, the same grid, or a different kernel, synchronization APIs such as `cooperative_groups::cluster` or `grid_group::sync()` or stream-order synchronization do not suffice to establish ordering for tensor map updates, that is, threads in these other thread blocks still need to acquire the tensor map proxy at the right scope before using the updated tensor map. If there are no intermediate modifications, the fence does not have to be repeated before each `cp.async.bulk` tensor instruction.

The fence and subsequent use of the tensor map is shown in the following example.

```

// Consumer of tensor map in global memory:
__global__ void consume_tensor_map(CUtensorMap* tensor_map) {
    // Fence acquire tensor map:
    ptx::n32_t<128> size_bytes;
    ptx::fence_proxy_tensormap_generic(ptx::sem_acquire, ptx::scope_sys, tensor_
    ↪map, size_bytes);
    // Safe to use tensor_map after fence.

    __shared__ uint64_t bar;
    __shared__ alignas(128) char smem_buf[4][128];

    if (threadIdx.x == 0) {
        // Initialize barrier
        ptx::mbarrier_init(&bar, 1);
        // Issue TMA request
        ptx::cp_async_bulk_tensor(ptx::space_cluster, ptx::space_global, smem_buf,
    ↪ tensor_map, {0, 0}, &bar);
        // Arrive on barrier. Expect 4 * 128 bytes.
        ptx::mbarrier_arrive_expect_tx(ptx::sem_release, ptx::scope_cta,
    ↪ ptx::space_shared, &bar, sizeof(smem_buf));
    }
    const int parity = 0;
    // Wait for load to have completed
    while (!ptx::mbarrier_try_wait_parity(&bar, parity)) {}

    // print items:
    printf("Got:\n\n");
    for (int j = 0; j < 4; ++j) {
        for (int i = 0; i < 128; ++i) {
            printf("%3d ", smem_buf[j][i]);
            if (i % 32 == 31) { printf("\n"); };
        }
        printf("\n");
    }
}

```

4.11.2.2.4 Creating a Template Tensor Map Value Using the Driver API

The following code creates a minimal tiled-type tensor map that can be subsequently modified on device.

```

CUtensorMap make_tensormap_template() {
    CUtensorMap template_tensor_map{};
    auto cuTensorMapEncodeTiled = get_cuTensorMapEncodeTiled();

    uint32_t dims_32          = 16;
    uint64_t dims_strides_64 = 16;
    uint32_t elem_strides     = 1;

    // Create the tensor descriptor.
    CUresult res = cuTensorMapEncodeTiled(
        &template_tensor_map, // CUtensorMap *tensorMap,

```

(continues on next page)

(continued from previous page)

```

CUtensorMapDataType::CU_TENSOR_MAP_DATA_TYPE_UINT8,
1,                // cuuint32_t tensorRank,
nullptr,          // void *globalAddress,
&dims_strides_64, // const cuuint64_t *globalDim,
&dims_strides_64, // const cuuint64_t *globalStrides,
&dims_32,         // const cuuint32_t *boxDim,
&elem_strides,    // const cuuint32_t *elementStrides,
CUtensorMapInterleave::CU_TENSOR_MAP_INTERLEAVE_NONE,
CUtensorMapSwizzle::CU_TENSOR_MAP_SWIZZLE_NONE,
CUtensorMapL2promotion::CU_TENSOR_MAP_L2_PROMOTION_NONE,
CUtensorMapFloatOOBfill::CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);

CU_CHECK(res);
return template_tensor_map;
}

```

4.11.2.2.5 Shared-Memory Bank Swizzling

By default, the TMA engine loads data to shared memory in the same order as it is laid out in global memory. However, this layout may not be optimal for certain shared memory access patterns, as it could cause shared memory bank conflicts. To improve performance and reduce bank conflicts, we can change the shared memory layout by applying a ‘swizzle pattern’.

Shared memory has 32 banks that are organized such that successive 32-bit words map to successive banks. Each bank has a bandwidth of 32 bits per clock cycle. When loading and storing shared memory, bank conflicts arise if the same bank is used multiple times within a transaction, resulting in reduced bandwidth. See [Shared Memory Access Patterns](#).

To ensure that data is laid out in shared memory in such a way that user code can avoid shared memory bank conflicts, the TMA engine can be instructed to ‘swizzle’ the data before storing it in shared memory and ‘unswizzle’ it when copying the data back from shared memory to global memory. The tensor map encodes the ‘swizzle mode’ indicating which swizzle pattern is used.

Example: Matrix Transpose

An example is the transpose of a matrix where data is mapped from row to column first access. The data is stored row major in global memory, but we want to also access it column wise in shared memory, which leads to bank conflicts. However, by using the 128 bytes ‘swizzle’ mode and new shared memory indices, they are eliminated.

In the example, we load an 8x8 matrix of type `int4`, stored as row major in global memory to shared memory. Then, each set of eight threads loads a row from the shared memory buffer and stores it to a column in a separate transpose shared memory buffer. This results in an eight-way bank conflict when storing. Finally, the transpose buffer is written back to global memory.

To avoid bank conflicts, the `CU_TENSOR_MAP_SWIZZLE_128B` layout can be used. This layout matches the 128 bytes row length and changes the shared memory layout in a way that both the column wise and row wise access don’t require the same banks per transaction.

The two tables, [Figure 51](#) and [Figure 52](#), below show the normal and the swizzled shared memory layout of the 8x8 matrix of type `int4` and its transpose matrix. The colors indicate which of the eight groups of four banks the matrix element is mapped to, and the margin row and margin column list the global memory row and column indices. The entries show the shared memory indices of the 16-byte matrix elements.

	0	1	2	3	4	5	6	7
0	[0,0]	[0,1]	[0,2]	[0,3]	[0,4]	[0,5]	[0,6]	[0,7]
1	[1,0]	[1,1]	[1,2]	[1,3]	[1,4]	[1,5]	[1,6]	[1,7]
2	[2,0]	[2,1]	[2,2]	[2,3]	[2,4]	[2,5]	[2,6]	[2,7]
3	[3,0]	[3,1]	[3,2]	[3,3]	[3,4]	[3,5]	[3,6]	[3,7]
4	[4,0]	[4,1]	[4,2]	[4,3]	[4,4]	[4,5]	[4,6]	[4,7]
5	[5,0]	[5,1]	[5,2]	[5,3]	[5,4]	[5,5]	[5,6]	[5,7]
6	[6,0]	[6,1]	[6,2]	[6,3]	[6,4]	[6,5]	[6,6]	[6,7]
7	[7,0]	[7,1]	[7,2]	[7,3]	[7,4]	[7,5]	[7,6]	[7,7]

Figure 51: In the shared memory data layout without swizzle, the shared memory indices are equivalent to the global memory indices. Per load instruction, one row is read and stored in a column of the transpose buffer. Since all matrix elements of the column in the transpose fall in the same bank, the store must be serialized, resulting in eight store transactions, giving an eight-way bank conflict per stored column.

	0	1	2	3	4	5	6	7
0	[0,0]	[0,1]	[0,2]	[0,3]	[0,4]	[0,5]	[0,6]	[0,7]
1	[1,1]	[1,0]	[1,3]	[1,2]	[1,5]	[1,4]	[1,7]	[1,6]
2	[2,2]	[2,3]	[2,0]	[2,1]	[2,6]	[2,7]	[2,4]	[2,5]
3	[3,3]	[3,2]	[3,1]	[3,0]	[3,7]	[3,6]	[3,5]	[3,4]
4	[4,4]	[4,5]	[4,6]	[4,7]	[4,0]	[4,1]	[4,2]	[4,3]
5	[5,5]	[5,4]	[5,7]	[5,6]	[5,1]	[5,0]	[5,3]	[5,2]
6	[6,6]	[6,7]	[6,4]	[6,5]	[6,2]	[6,3]	[6,0]	[6,1]
7	[7,7]	[7,6]	[7,5]	[7,4]	[7,3]	[7,2]	[7,1]	[7,0]

Figure 52: The shared memory data layout with CU_TENSOR_MAP_SWIZZLE_128B swizzle. One row is stored in a column, each matrix element is from a different bank for both the rows and columns, and so without any bank conflicts.

```
__global__ void kernel_tma(const __grid_constant__ CUtensorMap tensor_map) {
    // The destination shared memory buffer of a bulk tensor operation
    // with the 128-byte swizzle mode, it should be 1024 bytes aligned.
    __shared__ alignas(1024) int4 smem_buffer[8][8];
    __shared__ alignas(1024) int4 smem_buffer_tr[8][8];

    // Initialize shared memory barrier
    #pragma nv_diag_suppress static_var_with_dynamic_init
    __shared__ barrier bar;

    if (threadIdx.x == 0) {
        init(&bar, blockDim.x);
    }
    __syncthreads();

    barrier::arrival_token token;
    if (is_elected()) {
        // Initiate bulk tensor copy from global to shared memory,
        // in the same way as without swizzle.
        int32_t tensor_coords[2] = { 0, 0 };
        ptx::cp_async_bulk_tensor(
            ptx::space_shared, ptx::space_global,
            &smem_buffer, &tensor_map, tensor_coords,
            cuda::device::barrier_native_handle(bar));
        token = cuda::device::barrier_arrive_tx(bar, 1, sizeof(smem_buffer));
    }
}
```

(continues on next page)

(continued from previous page)

```

    } else {
        token = bar.arrive();
    }

    bar.wait(std::move(token));

    /* Matrix transpose
     * When using the normal shared memory layout, there are eight
     * 8-way shared memory bank conflict when storing to the transpose.
     * When enabling the 128-byte swizzle pattern and using the according
    ↪access pattern,
     * they are eliminated both for load and store. */
    for(int sidx_j = threadIdx.x; sidx_j < 8; sidx_j += blockDim.x){
        for(int sidx_i = 0; sidx_i < 8; ++sidx_i){
            const int swiz_j_idx = (sidx_i % 8) ^ sidx_j;
            const int swiz_i_idx_tr = (sidx_j % 8) ^ sidx_i;
            smem_buffer_tr[sidx_j][swiz_i_idx_tr] = smem_buffer[sidx_i][swiz_j_
    ↪idx];
        }
    }

    // Wait for shared memory writes to be visible to TMA engine.
    ptx::fence_proxy_async(ptx::space_shared);
    __syncthreads();

    /* Initiate TMA transfer to copy the transposed shared memory buffer back
    ↪to global memory,
     * it will 'unswizzle' the data. */
    if (is_elected()) {
        int32_t tensor_coords[2] = { x, y };
        ptx::cp_async_bulk_tensor(
            ptx::space_global, ptx::space_shared,
            &tensor_map, tensor_coords, &smem_buffer_tr);
        ptx::cp_async_bulk_commit_group();
        ptx::cp_async_bulk_wait_group_read(ptx::n32_t<0>());
    }

    // Destroy barrier
    if (threadIdx.x == 0) {
        (&bar)->~barrier();
    }
}

// ----- main -----
↪--

int main(){
    ...
    void* tensor_ptr = d_data;

    CUTensorMap tensor_map{};

```

(continues on next page)

(continued from previous page)

```

// rank is the number of dimensions of the array.
constexpr uint32_t rank = 2;
// global memory size
uint64_t size[rank] = {4*8, 8};
// global memory stride, must be a multiple of 16.
uint64_t stride[rank - 1] = {8 * sizeof(int4)};
// The inner shared memory box dimension in bytes, equal to the swizzle
→span.
uint32_t box_size[rank] = {4*8, 8};

uint32_t elem_stride[rank] = {1, 1};

// Create the tensor descriptor.
CUresult res = cuTensorMapEncodeTiled(
    &tensor_map,                // CUtensorMap *tensorMap,
    CUpTensorMapDataType::CU_TENSOR_MAP_DATA_TYPE_INT32,
    rank,                       // cuuint32_t tensorRank,
    tensor_ptr,                 // void *globalAddress,
    size,                       // const cuuint64_t *globalDim,
    stride,                     // const cuuint64_t *globalStrides,
    box_size,                   // const cuuint32_t *boxDim,
    elem_stride,                // const cuuint32_t *elementStrides,
    CUpTensorMapInterleave::CU_TENSOR_MAP_INTERLEAVE_NONE,
    // Using a swizzle pattern of 128 bytes.
    CUpTensorMapSwizzle::CU_TENSOR_MAP_SWIZZLE_128B,
    CUpTensorMapL2promotion::CU_TENSOR_MAP_L2_PROMOTION_NONE,
    CUpTensorMapFloat00Bfill::CU_TENSOR_MAP_FLOAT_00B_FILL_NONE
);

kernel_tma<<<1, 8>>>(tensor_map);
...
}

```

Remark. This example is supposed to show the use of swizzle and ‘as-is’ is not performant nor does it scale beyond the given dimensions.

Explanation. During data transfer, the TMA engine shuffles the data according to the swizzle pattern, as described in the following tables. These swizzle patterns define the mapping of the 16-byte chunks along the swizzle width to subgroups of four banks. It is of type `CUpTensorMapSwizzle` and has four options: none, 32 bytes, 64 bytes and 128 bytes. Note that the shared memory box’s inner dimension must be less than or equal to the span of the swizzle pattern.

The Swizzle Modes

As previously mentioned, there are four swizzle modes. The following tables show the different swizzle patterns, including the relation of the new shared memory indices. The tables define the mapping of the 16-byte chunks along the 128 bytes to eight subgroups of four banks.

Considerations. When applying a TMA swizzle pattern, it is crucial to adhere to specific memory requirements:

- **Global memory alignment:** Global memory must be aligned to 128 bytes.
- **Shared memory alignment:** For simplicity shared memory should be aligned according to the number of bytes after which the swizzle pattern repeats. When the shared memory buffer is

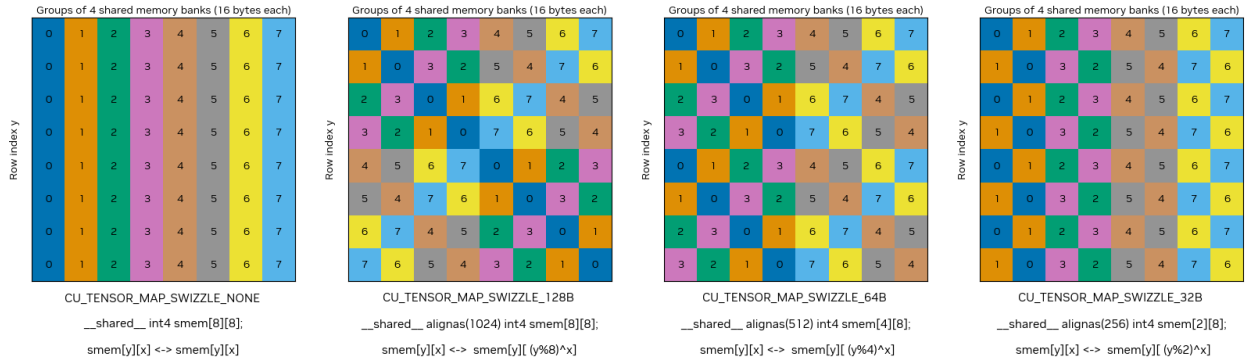


Figure 53: An Overview of TMA Swizzle Patterns

not aligned by the number of bytes by which the swizzle pattern repeats itself, there is an offset between the swizzle pattern and the shared memory. See [comment](#), below.

- **Inner dimension:** The inner dimension of the shared memory block must meet the size requirements specified in [Table 25](#). If these requirements are not met, the instruction is considered invalid. Additionally, if the swizzle width exceeds the inner dimension, ensure that the shared memory is allocated to accommodate the full swizzle width.
- **Granularity:** The granularity of swizzle mapping is fixed at 16 bytes. This means that data is organized and accessed in chunks of 16 bytes, which must be considered when planning memory layout and access patterns.

Swizzle Pattern Pointer Offset Computation. Here, we describe how to determine the offset between the swizzle pattern and the shared memory, when the shared memory buffer is not aligned by the number of bytes by which the swizzle pattern repeats itself. When using TMA, the shared memory is required to be aligned to 128 bytes. To find how many times the shared memory buffer relative to the swizzle pattern is shifted by that, apply the corresponding offset formula.

Table 24: Swizzle Pattern Pointer Offset Formula and Index Relation

Swizzle Mode	Offset Formula	Index Relation
CU_TENSOR_MAP_SWIZZLE_NONE	$(\text{reinterpret_cast}\langle\text{uintptr_t}\rangle(\text{smem_ptr})/128)\%8$	$\text{smem}[y][x] \leftrightarrow \text{smem}[y][((y+\text{offset})\%8)^x]$
CU_TENSOR_MAP_SWIZZLE_128B	$(\text{reinterpret_cast}\langle\text{uintptr_t}\rangle(\text{smem_ptr})/128)\%4$	$\text{smem}[y][x] \leftrightarrow \text{smem}[y][((y+\text{offset})\%4)^x]$
CU_TENSOR_MAP_SWIZZLE_64B	$(\text{reinterpret_cast}\langle\text{uintptr_t}\rangle(\text{smem_ptr})/128)\%2$	$\text{smem}[y][x] \leftrightarrow \text{smem}[y][((y+\text{offset})\%2)^x]$

In [Figure 53](#), this offset represents the initial row offset, thus, in the swizzle index calculation, it is added to the row index y. The following snippet shows how to access the swizzled shared memory in the CU_TENSOR_MAP_SWIZZLE_128B mode.

```
data_t* smem_ptr = &smem[0][0];
int offset = (reinterpret_cast<uintptr_t>(smem_ptr)/128)%8;
smem[y][((y+offset)%8)^x] = ...
```

Summary. The following [Table 25](#) summarizes the requirements and properties of the different swizzle

patterns for Compute Capability 9.

Table 25: Requirements and properties of the different swizzle patterns for Compute Capability 9

Pattern	Swizzle width	Shared box's inner dimension	Re-peats after	Shared memory alignment	Global memory alignment
CU_TENSOR_MAP_SWIZZ1	128 bytes	<=128 bytes	1024 bytes	128 bytes	128 bytes
CU_TENSOR_MAP_SWIZZ2	64 bytes	<=64 bytes	512 bytes	128 bytes	128 bytes
CU_TENSOR_MAP_SWIZZ3	32 bytes	<=32 bytes	256 bytes	128 bytes	128 bytes
CU_TENSOR_MAP_SWIZZ4 (default)				128 bytes	16 bytes

4.11.3. Using STAS

CUDA applications using *thread block clusters* may need to move small data elements between thread blocks within the cluster. STAS instructions (CC 9.0+, see [PTX documentation](#)) enable asynchronous data copies directly from registers to distributed shared memory. STAS is only exposed through a lower-level `cuda::ptx::st_async` API available in the `libcu++` library.

Dimensions. STAS supports copying 4, 8 or 16 bytes.

Source and destination. The only direction supported for asynchronous copy operations with STAS is from registers to distributed shared memory. The destination pointer needs to be aligned to 4, 8, or 16 bytes depending on the size of the data being copied.

Asynchronicity. Data transfers using STAS are *asynchronous* and are modeled as async thread operations (see [Async Thread and Async Proxy](#)). This allows the initiating thread to continue computing while the hardware asynchronously copies the data. *Whether the data transfer occurs asynchronously in practice is up to the hardware implementation and may change in the future.* The completion mechanisms that STAS operations can use to signal that they have completed are *shared memory barriers*.

In the following example, we show how to use STAS to implement a producer-consumer pattern within a thread-block cluster. This kernel creates a circular communication pipeline where 8 thread blocks are arranged in a ring, and each block simultaneously:

- Produces data for the next block in the sequence.
- Consumes data from the previous block in the sequence.

To implement this pattern, we need 2 shared memory barriers per thread block, one to notify the consumer block that the data has been copied to the shared memory buffer (filled) and one to notify the producer block that the buffer on the consumer is ready to be filled (ready).

CUDA C++ `cuda::ptx`

```

#include <cooperative_groups.h>
#include <cuda/barrier>
#include <cuda/ptx>

__global__ __cluster_dims__(8, 1, 1) void producer_consumer_kernel()
{
    using namespace cooperative_groups;
    using namespace cuda::device;
    using namespace cuda::ptx;
    using barrier_t = cuda::barrier<cuda::thread_scope_block>;

    auto cluster = this_cluster();

    #pragma nv_diag_suppress static_var_with_dynamic_init
    __shared__ int buffer[BLOCK_SIZE];
    __shared__ barrier_t filled;
    __shared__ barrier_t ready;

    // Initialize shared memory barriers.
    if (threadIdx.x == 0) {
        init(&filled, 1);
        init(&ready, BLOCK_SIZE);
    }

    // Sync cluster to ensure remote barriers are initialized.
    cluster.sync();

    // Define my own and my neighbor's ranks.
    int rk = cluster.block_rank();
    int rk_next = (rk + 1) % 8;
    int rk_prev = (rk + 7) % 8;

    // Get addresses of remote buffer we are writing to and remote barriers
    // of previous and next blocks.
    auto buffer_next = cluster.map_shared_rank(buffer, rk_next);
    auto bar_next = cluster.map_shared_rank(barrier_native_handle(filled),
    rk_next);
    auto bar_prev = cluster.map_shared_rank(barrier_native_handle(ready),
    rk_prev);

    int phase = 0;
    for (int it = 0; it < 1000; ++it) {

        // As producers, send data to our right neighbor.
        st_async(&buffer_next[threadIdx.x], rk, bar_next);

        if (threadIdx.x == 0) {
            // Thread 0 arrives on local barrier and indicates it expects to
            // receive a certain number of bytes.
            mbarrier_arrive_expect_tx(sem_release, scope_cluster, space_
            shared, barrier_native_handle(filled), sizeof(buffer));
        }

        // As consumers, wait on local barrier for data from left neighbor to
        // arrive.
        while (!mbarrier_try_wait_parity(barrier_native_handle(filled),

```

- ▶ Shared memory barriers are initialized by the first thread of each block. Barrier `filled` is initialized to 1 and barrier `ready` is initialized to the number of threads in the block.
- ▶ A cluster-wide synchronization is performed to ensure that all barriers are initialized before any thread starts communication.
- ▶ Each thread determines its neighbors' ranks and uses them to map the remote shared memory barriers and the remote shared memory buffer to write data to.
- ▶ In each iteration:
 1. As a producer, each thread sends data to its right neighbor.
 2. As a consumer, thread 0 arrives on the local `filled` barrier and indicates it expects to receive a certain number of bytes.
 3. As a consumer, each thread waits on the local `filled` barrier for data from the left neighbor to arrive.
 4. As a consumer, each thread uses the data to do something.
 5. As a consumer, each thread notifies the left neighbor that it is done with the data.
 6. As a producer, each thread waits on the local `ready` barrier until the right neighbor is ready to receive new data.

Note that for each barrier, we need to use the correct space. For mapped remote barriers, we need to use the `space_cluster` space, while for local barriers, we need to use the `space_shared` space.

4.12. Work Stealing with Cluster Launch Control

Dealing with problems of variable data and computation sizes is essential when developing CUDA applications. Traditionally, CUDA developers have used two main approaches to determine the number of kernel thread blocks to launch: *fixed work per thread block* and *fixed number of thread blocks*. Both approaches have their advantages and disadvantages.

Fixed Work per Thread Block: In this approach, the number of thread blocks is determined by the problem size, while the amount of work done by each thread block remains constant.

Key advantages of this approach:

- ▶ *Load balancing between SMs*

When thread block run-times exhibit variability and/or when the number of thread blocks is much larger than what the GPU can execute simultaneously (resulting in a low-tail effect), this approach allows the GPU scheduler to run more thread blocks on some SMs than others.

- ▶ *Preemption*

The GPU scheduler can start executing a *higher-priority kernel*, even if it is launched after a lower-priority kernel has already begun executing, by scheduling its thread blocks as thread blocks of the lower-priority kernel complete. It can then resume execution of the lower-priority kernel once the higher-priority kernel has finished executing.

Fixed Number of Thread Blocks: In this approach, often implemented as a block-stride or grid-stride loop, the number of thread blocks does not depend on the problem size. Instead, the amount of work done by each thread block is a function of the problem size. Typically, the number of thread blocks is based on the number of SMs on the GPU where the kernel is executed and the desired occupancy.

Key advantages of this approach:

► *Reduced thread block overheads*

This approach not only reduces amortized thread block launch latency but also minimizes the computational overhead associated with shared operations across all thread blocks. These overheads can be significantly higher than launch latency overheads.

For example, in convolution kernels, a prologue for calculating convolution coefficients – independent of the thread block index – can be computed fewer times due to the fixed number of thread blocks, thus reducing redundant computations.

Cluster Launch Control is a feature introduced in the NVIDIA Blackwell GPU architecture (compute capability 10.0) that aims to combine the benefits of the previous two approaches. It provides developers with more control over thread block scheduling by allowing them to cancel thread blocks or thread block clusters. This mechanism enables work stealing. Work stealing is a dynamic load-balancing technique in parallel computing where idle processors actively “steal” tasks from the work queues of busy processors, rather than wait for work to be assigned.

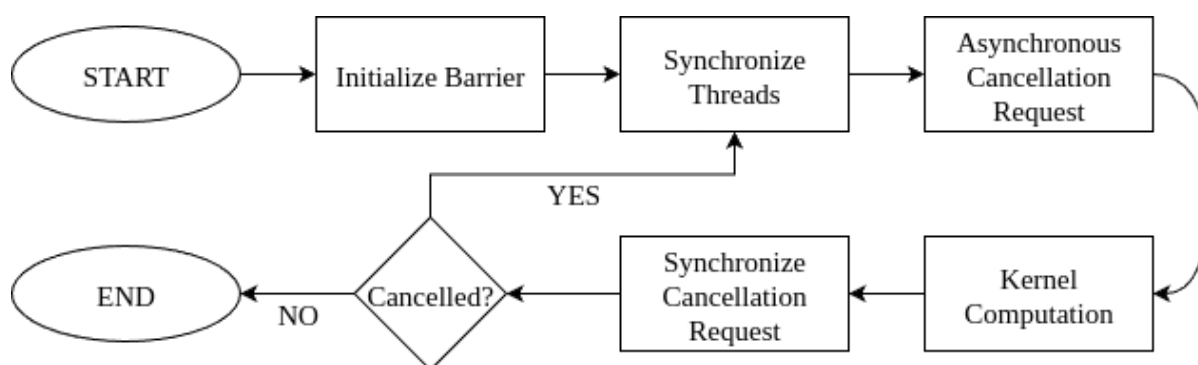


Figure 54: Cluster Launch Control Flow

With cluster launch control, a thread block attempts to cancel the launch of another thread block that has not started executing yet. If the cancellation request succeeds, it “steals” the other thread block’s work by using its index to perform the task. The cancellation will fail if there are no more thread block indices available or for other reasons, such as a higher-priority kernel being scheduled. In the latter case, if a thread block exits after a cancellation failure, the scheduler can start executing the higher-priority kernel, after which it will continue scheduling the remaining thread blocks of the current kernel for execution. The [figure](#) above presents the execution flow of this procedure.

The table below summarizes advantages and disadvantages of the three approaches:

	Fixed Work per Thread Block	Fixed Number of Thread Blocks	Cluster Launch Control
Reduced overheads	X	V	V
Preemption	V	X	V
Load balancing	V	X	V

4.12.1. API Details

Cancelling a thread block via the cluster launch control API is done asynchronously and synchronized using a shared memory barrier, following a programming pattern similar to *asynchronous data copies*.

The API, available through `libcu++`, provides:

- ▶ A request instruction that writes encoded cancellation results to a `__shared__` variable.
- ▶ Decoding instructions that extract success/failure status and the cancelled thread block index.

Note that cluster launch control operations are modeled as async proxy operations (see *Async Thread and Async Proxy*).

4.12.1.1 Thread Block Cancellation

The preferred way to use Cluster Launch Control is from a single thread, i.e., one request at a time.

The cancellation process involves five steps:

- ▶ **Setup Phase** (Steps 1-2): Declare and initialize cancellation result and synchronization variables.
- ▶ **Work-Stealing Loop** (Steps 3-5): Execute repeatedly to request, synchronize, and process cancellation results.

1. Declare variables for thread block cancellation:

```
__shared__ uint4 result; // Request result.
__shared__ uint64_t bar; // Synchronization barrier.
int phase = 0;           // Synchronization barrier phase.
```

2. Initialize shared memory barrier with a single arrival count:

```
if (cg::thread_block::thread_rank() == 0)
    ptx::mbarrier_init(&bar, 1);
__syncthreads();
```

3. Submit asynchronous cancellation request by a single thread and set transaction count:

```
if (cg::thread_block::thread_rank() == 0) {
    cg::invoke_one(cg::coalesced_threads(), [&]() {
        → ptx::clusterlaunchcontrol_try_cancel(&result, &bar);
        ptx::mbarrier_arrive_expect_tx(ptx::sem_relaxed, ptx::scope_cta,
        → ptx::space_shared, &bar, sizeof(uint4));
    })
}
```

Note

Since thread block cancellation is a uniform instruction, it is recommended to submit it inside *invoke_one* thread selector. This allows the compiler to optimize out the peeling loop.

4. Synchronize (complete) asynchronous cancellation request:

```
while (!ptx::mbarrier_try_wait_parity(&bar, phase))
    {}
phase ^= 1;
```

5. Retrieve cancellation status and cancelled thread block index:

```
bool success = ptx::clusterlaunchcontrol_query_cancel_is_canceled(result);
if (success) {
    // Don't need all three for 1D/2D thread blocks:
    int bx = ptx::clusterlaunchcontrol_query_cancel_get_first_ctaid_
    ↪x(result);
    int by = ptx::clusterlaunchcontrol_query_cancel_get_first_ctaid_
    ↪y(result);
    int bz = ptx::clusterlaunchcontrol_query_cancel_get_first_ctaid_
    ↪z(result);
}
```

6. Ensure visibility of shared memory operations between async and generic proxies, and protect against data races between iterations of the work-stealing loop.

4.12.1.2 Constraints on Thread Block Cancellation

The constraints are related to failed cancellation requests:

- Submitting another cancellation request after **observing** a previously failed request is *undefined behavior*.

In the two code examples below, assuming the first cancellation request fails, only the first example exhibits undefined behavior. The second example is correct because there is no observation between the cancellation requests:

Invalid code:

```
// First request:
ptx::clusterlaunchcontrol_try_cancel(&result0, &bar0);

// First request query:
[Synchronize bar0 code here.]
bool success0 = ptx::clusterlaunchcontrol_query_cancel_is_
    ↪canceled(result0);
assert(!success0); // Observed failure; second cancellation will be
    ↪invalid.

// Second request - next line is Undefined Behavior:
ptx::clusterlaunchcontrol_try_cancel(&result1, &bar1);
```

Valid code:

```
// First request:
ptx::clusterlaunchcontrol_try_cancel(&result0, &bar0);

// Second request:
ptx::clusterlaunchcontrol_try_cancel(&result1, &bar1);

// First request query:
[Synchronize bar0 code here.]
bool success0 = ptx::clusterlaunchcontrol_query_cancel_is_
    ↪canceled(result0);
assert(!success0); // Observed failure; second cancellation was valid.
```


- ▶ Retrieving the thread block index of a failed cancellation request is Undefined Behavior.
- ▶ Submitting a cancellation request from multiple threads is not recommended. It results in the cancellation of multiple thread blocks and requires careful handling, such as:
 - ▶ Each submitting thread must provide a unique `__shared__` result pointer to avoid data races.
 - ▶ If the same barrier is used for synchronization, the arrival and transaction counts must be adjusted accordingly.

4.12.2. Example: Vector-Scalar Multiplication

In the following subsections, we demonstrate work stealing through cluster launch control with a vector-scalar multiplication kernel. We show two variants of the same problem: one using thread blocks and one using thread block clusters.

4.12.2.1 Use-case: Thread Blocks

The three kernels below demonstrate the *Fixed Work per Thread Block*, *Fixed Number of Thread Blocks*, and *Cluster Launch Control* approaches for vector-scalar multiplication $\bar{v} := \alpha \bar{v}$.

- ▶ Fixed Work per Thread Block:

```
__global__
void kernel_fixed_work (float* data, int n)
{
    // Prologue:
    float alpha = compute_scalar();

    // Computation:
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
        data[i] *= alpha;
}

// Launch: kernel_fixed_work<<<(n + 1023) / 1024, 1024>>>(data, n);
```

- ▶ Fixed Number of Thread Blocks:

```
__global__
void kernel_fixed_blocks (float* data, int n)
{
    // Prologue:
    float alpha = compute_scalar();

    // Computation:
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    while (i < n) {
        data[i] *= alpha;
        i += blockDim.x * blockDim.x;
    }
}

// Launch: kernel_fixed_blocks<<<SM_COUNT, 1024>>>(data, n);
```

► Cluster Launch Control:

```

#include <cooperative_groups.h>
#include <cuda/ptx>

namespace cg = cooperative_groups;
namespace ptx = cuda::ptx;

__global__
void kernel_cluster_launch_control (float* data, int n)
{
    // Cluster launch control initialization:
    __shared__ uint4 result;
    __shared__ uint64_t bar;
    int phase = 0;

    if (cg::thread_block::thread_rank() == 0)
        ptx::mbarrier_init(&bar, 1);

    // Prologue:
    float alpha = compute_scalar(); // Device function not shown in this
    ↪code snippet.

    // Work-stealing loop:
    int bx = blockIdx.x; // Assuming 1D x-axis thread blocks.

    while (true) {
        // Protect result from overwrite in the next iteration,
        // (also ensure barrier initialization at 1st iteration):
        __syncthreads();

        // Cancellation request:
        if (cg::thread_block::thread_rank() == 0) {
            // Acquire write of result in the async proxy:
            ptx::fence_proxy_async_generic_sync_restrict(ptx::sem_acquire,
            ↪ ptx::space_cluster, ptx::scope_cluster);

            cg::invoke_one(cg::coalesced_threads(), [&]()
            ↪{ptx::clusterlaunchcontrol_try_cancel(&result, &bar);});
            ptx::mbarrier_arrive_expect_tx(ptx::sem_relaxed, ptx::scope_
            ↪cta, ptx::space_shared, &bar, sizeof(uint4));
        }

        // Computation:
        int i = bx * blockDim.x + threadIdx.x;
        if (i < n)
            data[i] *= alpha;

        // Cancellation request synchronization:
        while (!ptx::mbarrier_try_wait_parity(ptx::sem_acquire,
        ↪ptx::scope_cta, &bar, phase))
        {}
        phase ^= 1;
    }
}

```

(continues on next page)

(continued from previous page)

```

        // Cancellation request decoding:
        bool success = ptx::clusterlaunchcontrol_query_cancel_is_
        ↪ canceled(result);
        if (!success)
            break;

        bx = ptx::clusterlaunchcontrol_query_cancel_get_first_ctaid_x<int>
        ↪ (result);

        // Release read of result to the async proxy:
        ptx::fence_proxy_async_generic_sync_restrict(ptx::sem_release,
        ↪ ptx::space_shared, ptx::scope_cluster);
    }
}

// Launch: kernel_cluster_launch_control<<<(n + 1023) / 1024, 1024>>>(data,
↪ n);

```

4.12.2.2 Use-case: Thread Block Clusters

In the case of a *thread block clusters*, the thread block cancellation steps are the same as in a non-cluster setting, with minor adjustments. As in the non-cluster case, submitting a cancellation request from multiple threads **within a cluster** is not recommended, as this will attempt to cancel multiple clusters.

- ▶ The cancellation is submitted by a single cluster thread.
- ▶ The shared memory result of each cluster's thread block will receive the same (encoded) value of the cancelled thread block index (i.e., the result value is multicasted). The result received by all thread blocks corresponds to the local block index {0, 0, 0} within a cluster. Therefore, thread blocks within the cluster need to add the local block index.
- ▶ Synchronization is performed by each cluster's thread block using a local `__shared__` memory barrier. Barrier operations must be performed with the `ptx::scope_cluster` scope.
- ▶ Cancelling in the cluster case requires all the thread blocks to exist. A user can guarantee that all thread blocks are running by using `cg::cluster_group::sync()` from *sync* API.

The kernel below demonstrates the cluster launch control approach using thread block clusters.

```

#include <cooperative_groups.h>
#include <cuda/ptx>

namespace cg = cooperative_groups;
namespace ptx = cuda::ptx;

__global__ __cluster_dims__(2, 1, 1)
void kernel_cluster_launch_control (float* data, int n)
{
    // Cluster launch control initialization:
    __shared__ uint4 result;
    __shared__ uint64_t bar;
    int phase = 0;

```

(continues on next page)

(continued from previous page)

```

    if (cg::thread_block::thread_rank() == 0) {
        ptx::mbarrier_init(&bar, 1);
        ptx::fence_mbarrier_init(ptx::sem_release, ptx::scope_cluster); //
→CGA-level fence.
    }

    // Prologue:
    float alpha = compute_scalar(); // Device function not shown in this code
→snippet.

    // Work-stealing loop:
    int bx = blockIdx.x; // Assuming 1D x-axis thread blocks.

    while (true) {
        // Protect result from overwrite in the next iteration,
        // (also ensure all thread blocks have started at 1st iteration):
        cg::cluster_group::sync();

        // Cancellation request by a single cluster thread:
        if (cg::cluster_group::thread_rank() == 0) {
            // Acquire write of result in the async proxy:
            ptx::fence_proxy_async_generic_sync_restrict(ptx::sem_acquire,
→ptx::space_cluster, ptx::scope_cluster);

            cg::invoke_one(cg::coalesced_threads(), [&]() {
→{ptx::clusterlaunchcontrol_try_cancel_multicast(&result, &bar);});
        }

        // Cancellation completion tracked by each thread block:
        if (cg::thread_block::thread_rank() == 0)
            ptx::mbarrier_arrive_expect_tx(ptx::sem_relaxed, ptx::scope_
→cluster, ptx::space_shared, &bar, sizeof(uint4));

        // Computation:
        int i = bx * blockDim.x + threadIdx.x;
        if (i < n)
            data[i] *= alpha;

        // Cancellation request synchronization:
        while (!ptx::mbarrier_try_wait_parity(ptx::sem_acquire, ptx::scope_
→cluster, &bar, phase))
        {}
        phase ^= 1;

        // Cancellation request decoding:
        bool success = ptx::clusterlaunchcontrol_query_cancel_is_
→canceled(result);
        if (!success)
            break;

        bx = ptx::clusterlaunchcontrol_query_cancel_get_first_ctaid_x<int>

```

(continues on next page)

(continued from previous page)

```

→(result);
    bx += cg::cluster_group::block_index().x; // Add local offset.

    // Release read of result to the async proxy:
    ptx::fence_proxy_async_generic_sync_restrict(ptx::sem_release,
→ptx::space_shared, ptx::scope_cluster);
    }
}

// Launch: kernel_cluster_launch_control<<<(n + 1023) / 1024, 1024>>>(data, n);

```

4.13. L2 Cache Control

When a CUDA kernel accesses a data region in the global memory repeatedly, such data accesses can be considered to be persisting. On the other hand, if the data is only accessed once, such data accesses can be considered to be streaming.

Devices of compute capability 8.0 and above have the capability to influence persistence of data in the L2 cache, potentially providing higher bandwidth and lower latency accesses to global memory.

This functionality is exposed through two main APIs:

- The CUDA runtime API (starting with CUDA 11.0) provides programmatic control over L2 cache persistence.
- The `cuda::annotated_ptr` API in the `libcu++` library (starting with CUDA 11.5) annotates pointers in CUDA kernels with memory access properties to achieve a similar effect..

The following sections focus on the CUDA runtime API. For detailed information about the `cuda::annotated_ptr` approach, please refer to the [libcu++ documentation](#).

4.13.1. L2 Cache Set-Aside for Persisting Accesses

A portion of the L2 cache can be set aside to be used for persisting data accesses to global memory. Persisting accesses have prioritized use of this set-aside portion of L2 cache, whereas normal or streaming accesses to global memory can only utilize this portion of L2 when it is unused by persisting accesses.

The L2 cache set-aside size for persisting accesses may be adjusted, within limits:

```

cudaGetDeviceProperties(&prop, device_id);
size_t size = min(int(prop.l2CacheSize * 0.75), prop.
→persistingL2CacheMaxSize);
cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, size); /* set-aside 3/4 of
→L2 cache for persisting accesses or the max allowed*/

```

When the GPU is configured in Multi-Instance GPU (MIG) mode, the L2 cache set-aside functionality is disabled.

When using the Multi-Process Service (MPS), the L2 cache set-aside size cannot be changed by `cudaDeviceSetLimit`. Instead, the set-aside size can only be specified at start up of MPS server through the environment variable `CUDA_DEVICE_DEFAULT_PERSISTING_L2_CACHE_PERCENTAGE_LIMIT`.

4.13.2. L2 Policy for Persisting Accesses

An access policy window specifies a contiguous region of global memory and a persistence property in the L2 cache for accesses within that region.

The code example below shows how to set an L2 persisting access window using a CUDA Stream.

CUDA Stream Example

```
cudaStreamAttrValue stream_attribute;
↪ // Stream level attributes data structure
stream_attribute.accessPolicyWindow.base_ptr = reinterpret_cast<void*>(ptr);
↪ // Global Memory data pointer
stream_attribute.accessPolicyWindow.num_bytes = num_bytes;
↪ // Number of bytes for persistence access.

↪ // (Must be less than cudaDeviceProp::accessPolicyMaxWindowSize)
stream_attribute.accessPolicyWindow.hitRatio = 0.6;
↪ // Hint for cache hit ratio
stream_attribute.accessPolicyWindow.hitProp = cudaAccessPropertyPersisting;
↪ // Type of access property on cache hit
stream_attribute.accessPolicyWindow.missProp = cudaAccessPropertyStreaming;
↪ // Type of access property on cache miss.

//Set the attributes to a CUDA stream of type cudaStream_t
cudaStreamSetAttribute(stream, cudaStreamAttributeAccessPolicyWindow, &stream_
↪ attribute);
```

When a kernel subsequently executes in CUDA stream, memory accesses within the global memory extent [ptr..ptr+num_bytes) are more likely to persist in the L2 cache than accesses to other global memory locations.

L2 persistence can also be set for a CUDA Graph Kernel Node as shown in the example below:

CUDA GraphKernelNode Example

```
cudaKernelNodeAttrValue node_attribute; //
↪ Kernel level attributes data structure
node_attribute.accessPolicyWindow.base_ptr = reinterpret_cast<void*>(ptr); //
↪ Global Memory data pointer
node_attribute.accessPolicyWindow.num_bytes = num_bytes; //
↪ Number of bytes for persistence access. //

↪ (Must be less than cudaDeviceProp::accessPolicyMaxWindowSize)
node_attribute.accessPolicyWindow.hitRatio = 0.6; //
↪ Hint for cache hit ratio
node_attribute.accessPolicyWindow.hitProp = cudaAccessPropertyPersisting; //
↪ Type of access property on cache hit
node_attribute.accessPolicyWindow.missProp = cudaAccessPropertyStreaming; //
↪ Type of access property on cache miss.

//Set the attributes to a CUDA Graph Kernel node of type cudaGraphNode_t
cudaGraphKernelNodeSetAttribute(node,
↪ cudaKernelNodeAttributeAccessPolicyWindow, &node_attribute);
```

The hitRatio parameter can be used to specify the fraction of accesses that receive the hitProp